



Menoufia University
Faculty of Electronic Engineering
Department of Computer Science & Engineering

Graduation Project

Keep It Green

An AI-Powered Smart Recycling and Reward System

A graduation project submitted in partial fulfilment of the requirements for the degree of B.Sc. in Computer Science & Engineering

By

Yousef Badr
Eslam Medhat
Ahmed Anwer
Mohanad Ayman
Menna Osama
Shahd El Aswad

Supervisor
Dr. / [Supervisor Name]

Head of the Department
Prof. Dr. / [Head Name]

Dean
Prof. Dr. / [Dean Name]

Academic Year 2024 - 2025

Approval Sheet

This is to certify that the graduation project entitled “Keep It Green — An AI-Powered Smart Recycling and Reward System” has been carried out by Yousef Badr, Eslam Medhat, Ahmed Anwer, Mohanad Ayman, Menna Osama, and Shahd El Aswad, under supervision, in the Department of Computer Science & Engineering, Faculty of Electronic Engineering, Menoufia University, and is approved in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science & Engineering.

Supervisor:

Name: Signature:

Examiner (1):

Name: Signature:

Examiner (2):

Name: Signature:

Head of the Department:

Name: Signature:

Date:/...../ 2025

Abstract

Egypt generates on the order of one hundred million tonnes of solid waste each year and, despite rapid recent progress, still fails to recover a large fraction of its recyclable beverage containers. The evidence from deposit-return countries such as Germany and Norway — where container-return rates exceed ninety-five per cent — shows that the decisive missing ingredient is not public awareness but an immediate, tangible reward for returning each container, a reward layer that Egypt currently lacks because it has neither national deposit legislation nor the barcode infrastructure such schemes require.

This project, Keep It Green, supplies that missing reward layer using artificial intelligence rather than legislation. It couples an AI-powered reverse-vending machine with a mobile reward application, a web dashboard, an on-machine kiosk, and a secure cloud backend. A user pairs their phone with the machine using an ATM-style six-digit code; a camera on the machine runs a YOLOv8 object-detection model that classifies each inserted item as a plastic bottle or an aluminium can; an Arduino opens the correct sorting compartment; and reward points are credited to the paired user instantly through the authenticated backend, later redeemable for partner-vendor offers.

The detector was trained on a unified dataset of 11,897 real-box images fused from four public detection datasets and achieves a mean Average Precision (mAP@0.5) of 0.928 on a held-out test set — exceeding the 0.90 target — while running well within the one-second real-time budget. The complete system was deployed publicly over HTTPS and validated end-to-end with forty-eight automated checks (23 core, 12 administrative, 13 multi-user isolation), all passing. This document describes the motivation, analysis, design, implementation, testing, and deployment of every part of the system in detail.

Keywords: *reverse vending, object detection, YOLOv8, recycling incentives, .NET, Flutter, Arduino, Internet of Things, circular economy.*

ملخص — Abstract (Arabic)

يهدف هذا المشروع، «حافظ على الأخضر»، إلى معالجة تدني معدلات إعادة تدوير العبوات في مصر من خلال بناء نظام ذكي لاسترجاع العبوات ومكافأة المستخدم. يستخدم النظام آلة مزودة بكاميرا ونموذج ذكاء اصطناعي (YOLOv8) للتعرف على الزجاجات البلاستيكية وعلب الألمنيوم وفرزها آلياً، بينما يقترن هاتف المستخدم بالآلة عبر رمز مكون من ستة أرقام على غرار أجهزة الصراف الآلي، فتُضاف النقاط إلى حسابه فوراً عبر خادم آمن يمكن استبدالها لاحقاً بعروض من شركاء تجاريين.

حقق النموذج متوسط دقة قدره 0.928، وهو أعلى من الهدف المحدد (0.90)، مع أداء لحظي. وقد جرى التحقق من النظام بالكامل عبر ثمانٍ وأربعين اختباراً آلياً نجحت جميعها، ونُشر على الإنترنت بأمان ليتمكن أي شخص من تجربته.

Acknowledgements

We would like to express our sincere gratitude to our supervisor and to the staff of the Department of Computer Science & Engineering, Faculty of Electronic Engineering, Menoufia University, for their guidance and support throughout this project. We thank our families for their patience and constant encouragement, and we thank one another — each member of the team — for the dedication and collaboration that turned an idea into a working, deployed system.

Table of Contents

(In Microsoft Word, right-click this table and choose “Update Field” to fill in the page numbers.)

List of Figures

Figure 1.1	The Keep It Green reverse-vending machine (industrial-design render).....	11
Figure 1.2	High-level system architecture of Keep It Green.....	14
Figure 3.1	Use-case diagram of the Keep It Green system.....	29
Figure 4.1	The four-layer architecture of the backend.....	35
Figure 4.2	Sequence of a complete recycling session.....	36
Figure 4.3	Entity–relationship model of the database.....	37
Figure 5.1	The dataset unification pipeline, from four raw sources to an augmented training set.....	45
Figure 5.2	Class distribution, split sizes, and data provenance of the unified dataset.....	46
Figure 5.3	Distribution of source image resolutions; the median is about 481×640 pixels.....	47
Figure 5.4	Bounding-box geometry: the boxes are real, human-drawn annotations — only about 1% are near-full-frame.....	47
Figure 5.5	Correlogram of label and box statistics, used to check for spurious spatial biases.....	48
Figure 5.6	Sample training images overlaid with their ground-truth bounding boxes.....	49
Figure 5.7	Training and validation loss and metric curves over training.....	52
Figure 5.8	Per-class Average Precision at IoU 0.5; aluminium clears the 0.90 target and plastic sits just below at 0.87.....	52
Figure 5.9	Normalized confusion matrix on the validation set.....	54
Figure 5.10	Distribution of detection confidence on the test set, with the 0.65 rejection threshold marked.....	55
Figure 5.11	Model predictions on random unseen test images.....	56
Figure 6.1	Representative backend classes organized by layer.....	60
Figure 6.2	The JSON Web Token authentication flow.....	61
Figure 7.1	The Keep It Green sign-in screen (brand shared across app and web).....	66
Figure 8.1	Kiosk — the place-an-item state with the live camera view and detection frame.....	72
Figure 8.2	Kiosk — the reward-summary state shown when a session finishes.....	73
Figure 8.3	State machine of the kiosk interface.....	73
Figure 9.1	Front view of the machine enclosure (3D design).....	77
Figure 9.2	Interior of the plastic and aluminium sorting compartments.....	77
Figure 11.1	Deployment topology of the deployed system.....	85

List of Tables

Table 1.1	Project deliverables.....	13
Table 1.2	Team members and primary responsibilities.....	16
Table 2.1	Container-return and recycling rates in selected countries.....	18
Table 2.2	Comparison of recycling-incentive approaches.....	21
Table 3.1	Non-functional requirements.....	28
Table 3.2	Use-case specification — Recycle an Item.....	29
Table 3.3	Use-case specification — Redeem a Reward.....	29
Table 3.4	Use-case specification — Manage Machines.....	30
Table 3.5	Use-case specification — Pair with a Machine.....	30
Table 3.6	Use-case specification — Finish a Session.....	30
Table 3.7	Risk register (excerpt).....	32
Table 4.1	System components and their responsibilities.....	34
Table 4.2	The Otp (session) entity.....	38
Table 4.3	The Transaction entity.....	38
Table 4.4	The User entity.....	38
Table 4.5	The Machine entity.....	39
Table 4.6	The rewards entities (Vendor, Promo, Redemption).....	39
Table 5.1	Detection classes, rewards, and actions.....	44
Table 5.2	Composition of the unified dataset by source.....	44
Table 5.3	YOLOv8 size variants and the selection rationale.....	50
Table 5.4	Training hyperparameters.....	51
Table 5.5	Test-set detection metrics (held-out data).....	53
Table 6.1	Principal API endpoints by controller.....	62
Table 7.1	The application's screens and their purpose.....	67
Table 8.1	Web application routes (hash-based).....	71
Table 9.1	Principal hardware components (prototype).....	75
Table 9.2	The one-byte serial sorting protocol.....	76
Table 10.1	Core recycling-flow checks (23 assertions).....	81
Table 10.2	Administrative-flow checks (12 assertions).....	81
Table 10.3	Multi-user isolation checks (13 assertions).....	82
Table 10.4	Summary of automated end-to-end results.....	83
Table 11.1	Environment configuration of the deployed backend.....	86
Table 11.2	Deployment challenges and their resolutions.....	87
Table 11.3	Cost profile of the deployed system (prototype).....	88
Table 12.1	Objectives and their outcomes.....	89
Table 12.2	Keep It Green versus existing approaches (outcome view).....	90
Table 12.3	SWOT analysis of Keep It Green.....	92

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AP / mAP	Average Precision / mean Average Precision
API	Application Programming Interface
BCE	Binary Cross-Entropy
CIoU	Complete Intersection over Union (loss)
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CPU / GPU	Central / Graphics Processing Unit
CRUD	Create, Read, Update, Delete
DFL	Distribution Focal Loss
DRS	Deposit-Return Scheme
DTO	Data Transfer Object
EF Core	Entity Framework Core
FP / FN / TP	False Positive / False Negative / True Positive
FPS	Frames Per Second
HTTP(S)	Hypertext Transfer Protocol (Secure)
IoU	Intersection over Union
JWT	JSON Web Token
ML	Machine Learning
ONNX	Open Neural Network Exchange
OTP	One-Time Password (pairing code)
PET	Polyethylene Terephthalate
PWA	Progressive Web App
QR	Quick-Response (code)
REST	Representational State Transfer
RVM	Reverse-Vending Machine
SPA	Single-Page Application
UTC	Coordinated Universal Time
YOLO	You Only Look Once

Chapter 1 Introduction

This chapter introduces the Keep It Green project. It presents the context and the problem the project addresses, the motivation behind it, the aim and objectives, the scope and deliverables, an overview of the proposed solution, the main contributions of the work, the development methodology followed by the team, and finally the organization of the remainder of this document.

1.1 Overview

Waste management is one of the defining challenges of modern cities, and beverage containers made of plastic and aluminium are among the most visible and voluminous components of everyday waste. Both materials are technically recyclable and, in the case of aluminium, almost infinitely so; yet the rate at which they are actually recovered remains low in the absence of a system that makes recycling both convenient and rewarding for the individual. Keep It Green is a graduation project that addresses this gap directly by building an intelligent reverse-vending machine that identifies recyclable items automatically, rewards the depositor for each accepted item, and sorts the item into the correct compartment.

Unlike a traditional bin, which is a passive receptacle, a reverse-vending machine (RVM) is an active device: it accepts a used container, verifies it, and gives the depositor something in return. Commercial reverse-vending machines already underpin the highly successful deposit-return schemes of northern Europe, where container-return rates routinely exceed ninety-five per cent. Those machines, however, rely on two enabling conditions that do not exist in the Egyptian market: national deposit legislation that attaches a refundable value to every container, and standardized barcodes printed on the containers that the machine scans to identify them. Keep It Green removes both dependencies. It uses computer vision to recognize the material of the item directly from a camera image, so it can operate in a market with no deposit law and on containers that may be crushed, unlabelled, or otherwise unfamiliar.

The system is not a single device but an integrated ecosystem of five cooperating components: an artificial-intelligence machine that performs the detection and sorting; a cloud backend that manages users, sessions, points, and rewards; a mobile application through which a user pairs with the machine and collects rewards; a web application that provides the same capabilities in a browser; and an on-machine kiosk interface designed to be understood at a glance. Together

these components turn the simple act of returning a bottle or a can into an immediate, phone-based reward, and they do so through a single, secure, authenticated backend.



Figure 1.1: *The Keep It Green reverse-vending machine (industrial-design render).*

1.2 Problem Statement

The central problem this project addresses can be stated concisely: citizens have little practical incentive to recycle individual beverage containers, and the collection systems available to them are inconvenient, provide no immediate reward, and give no feedback on the impact of the individual's contribution. As a consequence, large quantities of valuable and fully recyclable material are lost to landfill and to the environment. This is a behavioural problem as much as a technical one — people are willing to recycle when it is easy and worthwhile, and they are unwilling when it is neither.

A second, more technical problem sits underneath the first. Automatically distinguishing recyclable materials at the point of disposal — reliably, in real time, at low cost, and without the barcodes that commercial machines depend upon — is a non-trivial engineering task. A

workable solution must recognize the target materials accurately, reject everything that is not a target material, respond within a second so that the user is not kept waiting, and do all of this on affordable, commodity hardware. Solving both problems together — the behavioural and the technical — is the challenge that defines this project.

1.3 Motivation

Three observations motivate the project. The first is the scale of the problem, which is both global and local. Worldwide, consumers purchase close to one million plastic bottles every minute, and only a small fraction of all plastic ever produced has been recycled. In Egypt, the country generates on the order of one hundred million tonnes of solid waste annually and, while the national recycling rate has improved markedly in recent years, a large fraction of recyclable material is still not recovered.

The second observation is that the solution is already proven elsewhere. Reward-based reverse vending, operating under deposit-return legislation, achieves container-return rates above ninety-five per cent wherever it is deployed at scale. The missing ingredient in markets like Egypt is therefore not public awareness — surveys consistently show that people care about the environment — but a direct, tangible, convenient reward for the act of returning a container. Keep It Green supplies exactly that missing reward layer.

The third observation is that the conditions for a mobile-first reward system are unusually favourable in Egypt. The country has a population of well over one hundred million people, a low median age, and near-universal mobile-phone penetration. A system that delivers rewards to a smartphone, rather than dispensing cash or printing a coupon, aligns naturally with how the target population already lives and transacts, and it does so without the cost and mechanical complexity of a cash-handling machine.

1.4 Aim and Objectives

The aim of the project is to design, build, deploy, and validate a complete smart-recycling reward system that makes recycling convenient and rewarding, using artificial intelligence to identify recyclable materials without reliance on deposit legislation or barcodes. To achieve this aim, the following objectives were defined:

1. To design and build a working reverse-vending machine that recognizes plastic bottles and aluminium cans automatically using a camera and an object-detection model.

2. To train an object-detection model that meets a target accuracy of at least 0.90 mean Average Precision at an intersection-over-union threshold of 0.5, while running fast enough for real-time use (under one second per item).
3. To handle non-recyclable and unfamiliar items gracefully by rejecting them, without requiring a dedicated training class for every possible object.
4. To develop a secure, layered backend that manages user accounts, machines, recycling sessions, reward points, and reward redemptions, exposing its functionality through a documented web API.
5. To design a safe pairing and session model that correctly attributes each recycled item to the correct user, even when many people use the same machine.
6. To build a mobile application through which users register, pair with a machine, watch points accrue in real time, and redeem rewards, and a web application and on-machine kiosk offering the same capabilities.
7. To integrate a physical sorting mechanism driven by a microcontroller.
8. To deploy the complete system to a public host and validate it end-to-end with automated tests.

1.5 Scope and Deliverables

The project delivers a functional, end-to-end prototype rather than a mass-manufactured product. It covers two reward classes — plastic bottles and aluminium cans — which together represent the highest-value, most-frequently-discarded beverage containers. Items that are neither are rejected rather than sorted. The reward economy (points, vendors, and promotional codes) is fully implemented in software; the physical sorting mechanism is implemented at prototype scale using a microcontroller and servo motors. Large-scale manufacturing, financial-payment integration, and sorting of additional materials are treated as future work in Chapter 13. Table 1.1 lists the concrete deliverables of the project.

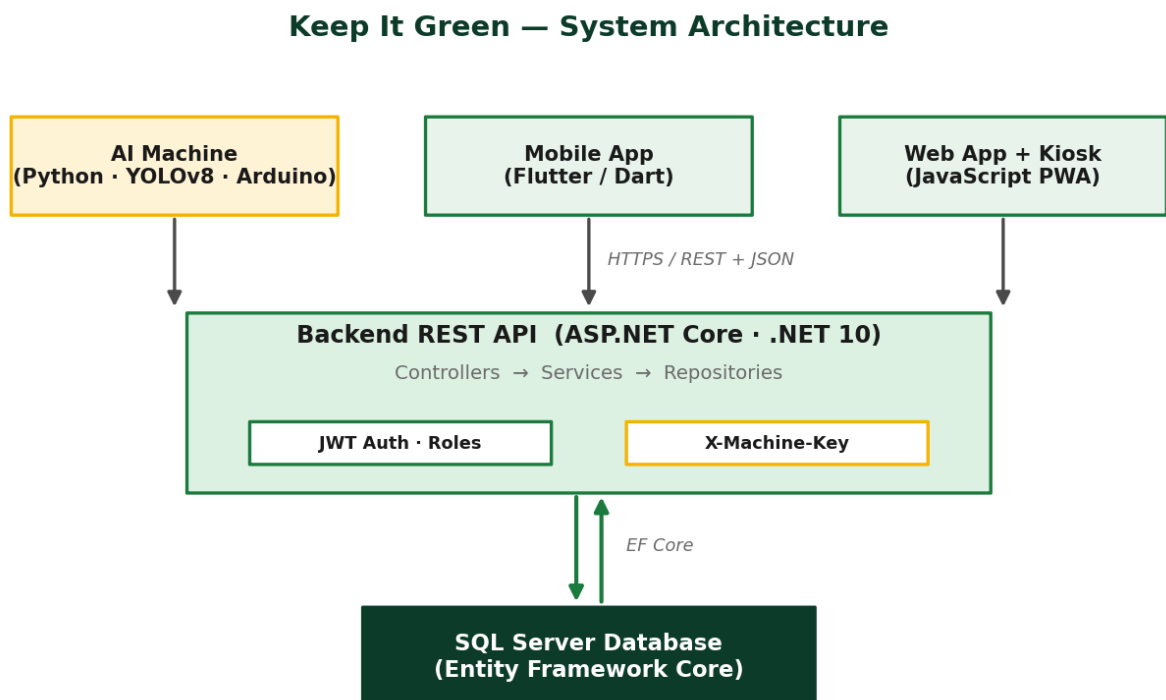
Table 1.1: *Project deliverables.*

#	Deliverable	Form
1	Trained object-detection model (plastic + aluminium)	.pt and .onnx model files
2	Machine control software (detection + sorting + session)	Python application
3	Backend web API with database	ASP.NET Core service, deployed
4	Mobile reward application	Installable Android package (APK)
5	Web application and administrator dashboard	Progressive Web App
6	On-machine kiosk interface	Web UI served by the backend

#	Deliverable	Form
7	Hardware sorting prototype	Arduino + servos + enclosure
8	Automated test suites	End-to-end test scripts

1.6 Overview of the Proposed Solution

The proposed solution links the five components through a single authenticated backend, following one strict design principle: no component ever touches the database directly. Every request — whether it originates from the machine, the phone, or a browser — passes through the backend web API and is authenticated before it can read or change any data. This concentrates all business rules and security in one place and makes the system safe to expose publicly. Figure 1.2 shows the high-level architecture; the components and their interactions are described in full in Chapter 4.



Design principle: no client ever touches the database directly — every request is authenticated at the API.

Figure 1.2: High-level system architecture of Keep It Green.

The user's experience is deliberately modelled on an automated teller machine. The machine displays a pairing code; the user pairs their phone to that code; the machine then attributes every item it detects to the paired user until the user finishes the session. This ATM-style model, detailed in Chapter 4, is what allows a single shared machine to serve many people safely and unambiguously.

1.7 Contributions

The main contributions of this work are the following:

- **A complete, integrated, and publicly deployed reverse-vending system** spanning artificial intelligence, cloud backend, mobile, web, and hardware — not a concept or a single module, but a working system that anyone can try.
- **A material-recognition detector** for plastic bottles and aluminium cans, trained on a unified dataset fused from four real-box detection sources, that exceeds the accuracy target while running in real time on commodity hardware.
- **An open-set rejection scheme** based on confidence thresholding, which allows the machine to reject non-recyclable items reliably without a dedicated “other” class that would otherwise dominate and bias training.
- **An ATM-style pairing and session model** that safely supports many simultaneous users on shared machines, verified by dedicated isolation testing.

1.8 Development Methodology

The project was developed using an iterative, incremental methodology inspired by Agile practice. Rather than attempting to specify and build the entire system in a single pass, the team worked in short cycles, each of which produced a working increment of the system that could be demonstrated and tested. Early cycles established the backend data model and the core recycling flow; later cycles added the artificial-intelligence detector, the mobile and web front-ends, the hardware integration, and finally public deployment and hardening. This approach allowed design decisions to be validated against a running system continuously, and it made it possible to detect and correct integration problems early, when they were cheap to fix. Version control, a shared backend, and automated end-to-end tests supported the parallel work of the team members, whose responsibilities spanned artificial intelligence and hardware, mobile development, front-end development, and back-end development.

1.9 Organization of the Book

The remainder of this document is organized as follows. Chapter 2 reviews the relevant background and related work, covering the waste problem, deposit-return schemes, reverse-vending machines, computer vision for waste sorting, and the object-detection techniques used. Chapter 3 presents the requirements engineering and analysis, including functional and non-functional requirements, use-case models, and feasibility and risk analyses. Chapter 4 describes the overall system architecture and design, including the database schema, the API, the security

design, and the session model. Chapter 5 details the artificial-intelligence module in depth. Chapters 6, 7, 8, and 9 cover the backend, the mobile application, the web and kiosk interfaces, and the hardware respectively. Chapter 10 presents the testing and validation strategy and its results. Chapter 11 covers deployment and operations. Chapter 12 discusses the results, and Chapter 13 concludes and outlines directions for future work. A set of appendices provides the full API reference, key code listings, the database schema, an installation and user manual, the complete test-case catalogue, and the hardware bill of materials.

1.10 The Team and Its Responsibilities

The project was carried out by a team of six students whose responsibilities were divided along the natural seams of the system, so that each major component had a clear owner while the interfaces between components were agreed and tested jointly. Table 1.2 records the division of responsibilities. The artificial-intelligence and hardware work — data, model, machine software, and the Arduino — formed the largest effort and was shared among three members; the mobile, front-end, and back-end responsibilities were each led by a dedicated member. Integration, testing, and deployment were shared across the team, reflecting the principle that the value of the project lies in the working whole.

Table 1.2: *Team members and primary responsibilities.*

Member	Primary responsibility
Yousef Badr	Team lead · AI and hardware · machine integration
Eslam Medhat	Artificial intelligence · dataset and model
Ahmed Anwer	Artificial intelligence · training and evaluation
Mohanad Ayman	Mobile application (Flutter)
Menna Osama	Front-end / web application
Shahd El Aswad	Back-end / API and database

1.11 Significance of the Project

The significance of Keep It Green lies in the way it combines a social objective with a technical one. Socially, it addresses a real and pressing problem — the low recovery of recyclable containers in a country actively seeking to raise its recycling rate — with an approach that is proven to change behaviour, namely an immediate reward for the individual. Technically, it demonstrates that this reward layer can be delivered without the deposit legislation and barcode infrastructure that formal schemes depend upon, by substituting affordable artificial intelligence for that infrastructure. The project therefore matters not only as an academic

exercise but as a demonstration of a deployable model for improving recycling in emerging markets.

For the students who built it, the project is also significant as a complete engineering experience. It required specifying, designing, implementing, integrating, testing, and deploying a system that spans machine learning, cloud services, mobile and web development, and embedded hardware — the full breadth of a modern software system — and doing so as a coordinated team. The result is evidence that the team can carry an ambitious idea all the way from concept to a working, deployed, and validated product.

Chapter 2 Background and Literature Review

This chapter establishes the context and the technical foundations for the project. It first examines the scale of the waste problem globally and in Egypt, and the properties and value of the two target materials. It then reviews deposit-return schemes and the commercial reverse-vending machines that implement them, together with app-based and informal alternatives, in order to position Keep It Green relative to existing solutions. Finally, it surveys the computer-vision and object-detection techniques on which the machine's intelligence is built, including the YOLO family of models and the metrics used to evaluate them, and closes with a gap analysis that motivates the design choices made in later chapters.

2.1 The Global and Local Waste Problem

The consumption of single-use beverage containers has reached an extraordinary scale. Globally, consumers purchase on the order of one million plastic bottles every minute, which amounts to roughly half a trillion bottles each year, and the trend has been rising. Despite decades of awareness campaigns, only about nine per cent of all the plastic ever produced has been recycled; the majority has been landfilled, incinerated, or has escaped into the natural environment, where it persists for centuries and fragments into microplastics. Aluminium tells a more encouraging but still incomplete story: it is infinitely recyclable without loss of quality, and its global recycling rate is considerably higher than that of plastic, yet a substantial fraction of aluminium cans is still discarded rather than recovered, representing both an environmental loss and a waste of the very large amount of energy embodied in primary aluminium production.

In Egypt, the picture reflects both the challenge and a growing momentum to address it. The country generates approximately one hundred million tonnes of solid waste per year. The national municipal-waste recycling rate rose from roughly ten per cent in 2018 to about thirty-seven per cent in 2024, and the government has publicly targeted a rate of sixty per cent by 2027. These figures indicate both the size of the gap that remains and a policy environment that is increasingly receptive to solutions that improve recovery rates. Table 2.1 places the local situation in international context.

Table 2.1: *Container-return and recycling rates in selected countries.*

Country / System	Rate	Mechanism
Germany (Pfand)	~98% container return	National deposit-return, ~135,000 return points

Country / System	Rate	Mechanism
Norway (Infinitum)	~97% container return	National deposit-return, ~3,500 machines
Egypt (municipal)	~37% recycled (2024)	Mixed formal/informal; target 60% by 2027
Global plastic	~9% ever recycled	Fragmented, largely without deposits

2.2 The Target Materials: PET Plastic and Aluminium

The two materials targeted by this project are chosen deliberately, because they are simultaneously the most common beverage containers and the most valuable to recover. Polyethylene terephthalate (PET) is the clear, lightweight plastic used for most water and soft-drink bottles. It is fully recyclable into new bottles, textile fibre, and packaging, and recovered PET commands a meaningful market price. Aluminium, used for beverage cans, is even more attractive from a circular-economy standpoint: recycling aluminium uses only about five per cent of the energy required to produce it from bauxite ore, and the material can be recycled indefinitely, so a recovered can carries a high intrinsic value. The considerable difference in per-tonne value between the two materials is one reason the system distinguishes them and awards different point values, and it is also part of the economic argument for the machine, which is discussed in Chapter 12.

From a computer-vision standpoint, the two materials present distinct visual signatures. PET bottles are typically transparent or lightly tinted, cylindrical, and deformable, with visible caps and labels; aluminium cans are opaque, metallic, and reflective, with a characteristic ridged top. These differences are what make automatic visual discrimination feasible, but the same deformability and reflectivity also introduce the variability — crushing, glare, partial labels — that the detection model must be robust to. This tension between distinctiveness and variability is a recurring theme of Chapter 5.

2.3 Deposit-Return Schemes

A deposit-return scheme (DRS) attaches a small, fully refundable deposit to every in-scope beverage container at the point of sale. The consumer reclaims the deposit by returning the empty container, typically to a reverse-vending machine located in a supermarket or public space. Because the deposit is the consumer's own money, the incentive to return the container is direct and immediate, and this simple mechanism is the most effective recycling intervention known. Germany's Pfand system achieves container-return rates above ninety-eight per cent across roughly one hundred and thirty-five thousand return points; Norway's system, operated by Infinitum, achieves about ninety-seven per cent through some three and a half thousand

machines. Several other countries operate comparable schemes with similarly high recovery rates.

The lesson these systems teach is the single most important premise of this project: when returning a container yields an immediate, tangible reward, return rates approach saturation. Egypt, however, has no national deposit-return legislation, and therefore no reward layer for the individual consumer. Keep It Green is designed precisely to supply that missing reward layer using technology rather than legislation — substituting a points-based reward funded by partner vendors for the legislated cash deposit, and substituting artificial-intelligence recognition for the barcode-and-deposit infrastructure that a formal DRS requires.

2.4 Reverse-Vending Machines and Commercial Solutions

Commercial reverse-vending is a mature industry dominated by specialist manufacturers, of which TOMRA is the best known, operating tens of thousands of machines worldwide. These machines are accurate, robust, and capable of handling high throughput, but they are engineered around the assumptions of a formal deposit scheme. They identify containers primarily by scanning the printed barcode and matching it against a national registry of registered products; they depend on the existence of a deposit to fund their operation; and they are capital-intensive, with per-unit costs far beyond the reach of an informal or emerging-market deployment. A machine that depends on a barcode also fails on a container whose label is missing, damaged, or not in the national registry — a common situation outside a mature DRS.

These characteristics define the niche that Keep It Green occupies. By recognizing the material of the item visually rather than reading a barcode, the machine becomes independent of any national product registry and can accept crushed or unlabelled containers. By funding rewards through partner vendors rather than a legislated deposit, it can operate where no deposit law exists. And by building on commodity hardware — a laptop-class computer, an ordinary webcam, and an inexpensive microcontroller — it targets a small fraction of the cost of a commercial unit. The trade-off is that visual recognition is inherently probabilistic and must be engineered carefully for accuracy and for the safe rejection of non-target items, which is the subject of Chapter 5.

2.5 App-Based and Informal Models

Between the extremes of the formal DRS machine and the ordinary bin lie several intermediate models. App-based schemes such as Bower, popular in Scandinavia, reward users for scanning a container's barcode with their phone; they are inexpensive to operate but require a clean, scannable label and a product database, and they perform no physical collection or sorting — the user is trusted to recycle the container afterwards. Informal collection, exemplified in Egypt by the long-established Zabaleen community and by various local buy-back arrangements, recovers a significant quantity of material and provides livelihoods, but it is unstructured, offers no standardized or convenient user experience, and provides no data or feedback. Keep It Green draws lessons from both: it adopts the phone-based reward of the app model while adding the physical verification and sorting of the machine model, and it formalizes the reward into a consistent, measurable, data-rich experience. Table 2.2 summarizes the comparison.

Table 2.2: *Comparison of recycling-incentive approaches.*

Approach	Reward	Physical sorting	Needs barcode/law	Cost
Commercial RVM (TOMRA)	Cash deposit	Yes	Yes (both)	Very high
App (Bower)	Points/cash	No	Barcode	Low
Informal (Zabaleen)	Cash (buy-back)	Manual	No	Low
Ordinary bin	None	No	No	Low
Keep It Green	Points → vendor rewards	Yes (AI)	Neither	Low–moderate

2.6 Computer Vision for Waste Sorting

The application of computer vision to waste sorting has grown rapidly as deep learning has matured. Research and industrial systems have applied image classification and object detection to distinguish waste categories — plastic, metal, glass, paper, and organic — on conveyor belts in materials-recovery facilities and, more recently, at the point of disposal. Convolutional neural networks, trained on labelled images of waste, have repeatedly been shown to outperform earlier approaches based on hand-engineered features such as colour histograms or edge descriptors, because they learn hierarchical visual features directly from data. Public datasets such as TrashNet, and more recent object-detection datasets carrying real bounding boxes, have made this research accessible, and it is from such detection sources that the dataset used in this project is assembled. The consensus of this body of work is that material-level recognition of clean, well-presented items is now highly reliable, and that the remaining challenges lie in robustness to real-world variability (deformation, occlusion,

lighting) and in the safe rejection of items outside the trained categories — precisely the issues this project confronts.

2.7 Object Detection Fundamentals

Object detection is the computer-vision task of simultaneously locating and classifying every object of interest within an image. For each object, a detector outputs a bounding box (the rectangular region containing the object), a class label, and a confidence score expressing how certain the model is. Detection is therefore more informative than plain image classification, which assigns a single label to a whole image, and this extra information — where the item is and how confident the model is — is exactly what the machine needs in order to confirm a stable detection and to reject uncertain ones.

Modern detectors fall into two broad families. Two-stage detectors, such as the R-CNN family, first propose candidate regions and then classify each region; they are typically very accurate but slower. One-stage detectors, such as SSD and the YOLO family, perform localization and classification in a single pass of the network; they are typically faster, which makes them the natural choice for real-time applications. Because Keep It Green must confirm an item within a second on modest hardware, a one-stage detector is the appropriate choice, and the YOLO family was selected for its combination of speed, accuracy, and mature, well-supported tooling.

2.7.1 Convolutional Neural Networks

The engine inside every modern detector is the convolutional neural network (CNN). A CNN processes an image through a stack of layers, each of which applies many small learnable filters that slide across their input and respond to local visual patterns. The earliest layers learn to detect simple features such as edges and colours; deeper layers combine these into textures, then parts, then whole-object concepts, building a hierarchy of increasingly abstract representations. Interleaved pooling operations progressively reduce spatial resolution while retaining the most salient information, which gives the network a degree of invariance to the exact position and scale of an object. Crucially, the filters are not hand-designed but learned from data by back-propagation, so a CNN discovers for itself which visual features best distinguish the target classes. This ability to learn features directly is why CNNs so decisively outperform earlier methods based on hand-crafted descriptors, and it is the foundation on which the YOLOv8 backbone of Chapter 5 is built.

2.7.2 Transfer Learning

Training a deep CNN from scratch requires very large datasets and long training times, because the network must learn all of its features from nothing. Transfer learning sidesteps this by starting from a network already trained on a large, general dataset and then adapting it to a new, narrower task. The pre-trained network has already learned generic visual features — edges, textures, shapes — that are useful for almost any vision problem, so only the higher, task-specific layers need to be substantially retrained. Transfer learning therefore lets a small dataset yield a highly accurate model, dramatically reduces training cost, and is standard practice in applied computer vision. It is the technique that allows this project's detector, described in Chapter 5, to reach commercial-grade accuracy from only a few thousand images.

2.8 The YOLO Family of Models

YOLO — “You Only Look Once” — is a family of one-stage object detectors first introduced in 2016 and refined through many subsequent versions. The defining idea is to treat detection as a single regression problem, dividing the image into a grid and predicting bounding boxes and class probabilities for the whole image in one forward pass, rather than running a classifier at many locations and scales. This makes YOLO exceptionally fast while remaining accurate. Successive versions have improved the backbone network, the feature-aggregation neck, and the detection head; introduced anchor boxes and later removed them again in favour of anchor-free prediction; and steadily improved the balance between speed and accuracy.

This project uses YOLOv8, a modern, widely-adopted version that is anchor-free and provides a spectrum of model sizes — nano, small, medium, large, and extra-large — trading accuracy against computational cost. The small variant (YOLOv8s) was chosen as the best balance for deployment on a laptop-class CPU, a decision justified in detail in Chapter 5. YOLOv8's architecture, comprising a CSPDarknet backbone, a path-aggregation feature-pyramid neck, and a decoupled detection head, is also described there.

2.9 Evaluation Metrics for Object Detection

Evaluating a detector requires metrics that capture both localization and classification quality. The foundation is Intersection over Union (IoU), which measures the overlap between a predicted box and the ground-truth box as the ratio of the area of their intersection to the area of their union:

$$IoU = \text{area}(\text{prediction} \cap \text{ground-truth}) / \text{area}(\text{prediction} \cup \text{ground-truth})$$

A prediction is counted as correct (a true positive) if its class is right and its IoU with a ground-truth box exceeds a chosen threshold, commonly 0.5. From the counts of true positives (TP), false positives (FP), and false negatives (FN), precision and recall are defined as:

$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN)$$

Precision measures how many of the model's detections are correct (few false alarms), while recall measures how many of the real objects the model found (few misses). Varying the confidence threshold traces out a precision–recall curve, and the area under this curve for a given class is its Average Precision (AP). Averaging AP over all classes gives the mean Average Precision (mAP), the standard headline metric for detectors. Two common forms are reported in this document: mAP@0.5, which uses a single IoU threshold of 0.5, and the stricter mAP@0.5:0.95, which averages mAP over IoU thresholds from 0.5 to 0.95 in steps of 0.05 and therefore rewards precise localization. These metrics and their formulas underpin the results presented in Chapter 5.

2.10 Embedded Systems and Actuation

The intelligence of the machine must ultimately produce a physical action: opening the correct compartment so that the item is sorted. This is handled by an embedded microcontroller. A microcontroller is a small, inexpensive computer-on-a-chip that reads inputs and drives outputs in real time; the Arduino platform, used in this project, is a widely-adopted, low-cost development board with extensive library support that makes it straightforward to drive servo motors. In Keep It Green, the detection software running on the machine's computer communicates with the Arduino over a simple serial link, sending a single byte that tells the microcontroller which compartment to open. This division of labour — heavy computation on the computer, precise real-time actuation on the microcontroller — is a standard and robust pattern in embedded and Internet-of-Things design, and it keeps the hardware simple, cheap, and reliable, as detailed in Chapter 9.

2.11 Summary and Gap Analysis

The literature and the existing solutions together define a clear gap. Deposit-return schemes prove that immediate rewards drive near-total container return, but they require national legislation and barcode infrastructure that Egypt lacks. Commercial reverse-vending machines implement those schemes accurately but at high cost and with a hard dependency on barcodes and deposits. App-based schemes are cheap but perform no physical verification or sorting, and informal collection is effective but unstructured. Meanwhile, computer vision has matured

to the point where material-level recognition of beverage containers is reliable on commodity hardware, provided the problem of rejecting out-of-category items is handled with care. Keep It Green sits precisely in this gap: it delivers the immediate, phone-based reward that drives behaviour, adds physical verification and sorting through low-cost artificial intelligence and hardware, and depends on neither deposit legislation nor barcodes. The chapters that follow describe how this system was specified, designed, built, tested, and deployed.

2.12 Alignment with the Sustainable Development Goals

The project is directly aligned with several of the United Nations Sustainable Development Goals, which situate it within a global agenda rather than a purely local one. It contributes to Goal 12, Responsible Consumption and Production, by increasing the recovery and recycling of beverage containers and by making the circular use of materials tangible and rewarding for the individual. It contributes to Goal 11, Sustainable Cities and Communities, by providing an accessible mechanism that keeps recyclable material out of the urban waste stream. And it contributes to Goal 13, Climate Action, indirectly but meaningfully, because recycling — especially of aluminium, which requires only a small fraction of the energy of primary production — reduces the greenhouse-gas emissions associated with manufacturing new material. By attaching an immediate reward to recycling, the system addresses the behavioural gap that these goals ultimately depend on closing.

Chapter 3 System Requirements and Analysis

This chapter captures what the system must do and the qualities it must exhibit, before any consideration of how it is built. It describes the approach taken to gather requirements, identifies the stakeholders and actors, sets out the functional and non-functional requirements, models the principal use cases in detail, expresses the requirements as user stories, records the constraints and assumptions under which the system operates, and closes with a feasibility study and a risk analysis.

3.1 Requirements Elicitation

Because the project is a novel product rather than a replacement for an existing system, requirements were elicited through a combination of techniques. The team studied analogous systems — commercial reverse-vending machines, recycling-reward applications, and deposit-return schemes — to understand established expectations and to identify their limitations. It analysed the target context in Egypt, including the absence of deposit legislation and the prevalence of smartphones, to determine which assumptions of the analogous systems did and did not hold. And it proceeded iteratively: each development cycle produced a working increment that was demonstrated and critiqued, and the resulting feedback refined the requirements for the next cycle. The requirements below are the consolidated result of this process.

3.2 Stakeholders and Actors

The system serves three principal actors, each with distinct goals. The End User recycles items and earns and redeems reward points through the mobile or web application. The Administrator manages the operational entities of the system — machines, partner vendors, and promotional rewards — and views recycling reports. The Machine is a software actor: an autonomous agent that starts recycling sessions, detects and sorts items, and posts transactions to the backend on behalf of the paired user. Beyond these direct actors, the wider stakeholders include partner vendors who fund the rewards, the institutions that host machines, and the environment and society that benefit from higher recovery rates.

3.3 Functional Requirements

The functional requirements specify the behaviour the system must provide. They are grouped by actor for clarity.

3.3.1 End-User Requirements

9. The system shall allow a user to register with an email address and password, and to log in securely.
10. The system shall allow a logged-in user to pair with a machine by scanning its QR code or entering its six-digit pairing code.
11. The system shall display a live session view in which the user's points update in real time as items are accepted.
12. The system shall credit reward points to the paired user for each accepted item: ten points for a plastic bottle and fifteen for an aluminium can.
13. The system shall allow a user to finish a session, after which the machine returns to its idle state.
14. The system shall present a catalogue of vendor rewards and allow a user to redeem points for a reward.
15. The system shall reveal a reward's promotional code only after the user has redeemed it.
16. The system shall provide each user with a history of their recycling transactions and redemptions, shown in local (Cairo) time.
17. The system shall display the user's current point balance at all times.

3.3.2 Machine Requirements

18. The machine shall request a new session from the backend and display the returned pairing code as both a QR code and six digits.
19. The machine shall detect and classify an inserted item as a plastic bottle or an aluminium can using the object-detection model.
20. The machine shall reject any item that is not confidently recognized as a target material.
21. The machine shall require a stable detection before accepting an item, to prevent misfires and double-counting.
22. The machine shall drive the correct sorting compartment for the accepted item.
23. The machine shall submit each accepted detection to the backend, authenticated with its secret machine key.
24. The machine shall detect that a session has ended and return to its idle state.

3.3.3 Administrator Requirements

25. The system shall allow an administrator to add, enable, disable, and delete machines.
26. The system shall allow an administrator to create partner vendors and promotional codes with a point cost.
27. The system shall allow an administrator to view the transaction report for any machine.
28. The system shall restrict all administrative functions to users holding the administrator role.

3.4 Non-Functional Requirements

Non-functional requirements specify the quality attributes the system must exhibit. They are summarized in Table 3.1 and are treated as first-class requirements: several of them (real-time performance, detection accuracy, and session correctness) are among the hardest and most important aspects of the whole project.

Table 3.1: *Non-functional requirements.*

ID	Attribute	Requirement
NFR-1	Performance	An item shall be detected and confirmed in under one second.
NFR-2	Accuracy	The detection model shall achieve $mAP@0.5 \geq 0.90$ on unseen data.
NFR-3	Security	All access shall be authenticated; roles separated; machine requests signed with a secret key; transport encrypted (HTTPS).
NFR-4	Reliability	Each accepted item shall be credited to exactly one user; no double-counting; misfires rejected.
NFR-5	Usability	The machine screen shall be ATM-simple; the app mobile-first; times shown in Cairo local time.
NFR-6	Portability	The backend shall be reachable from any device; the app installable on Android.
NFR-7	Scalability	A machine shall support many users; codes shall be globally unique.
NFR-8	Maintainability	Code shall be layered and testable, with automated end-to-end tests.

3.5 Use-Case Modelling

The principal interactions between the actors and the system are captured as use cases. Figure 3.1 shows the use-case diagram; the specifications that follow describe the most important use cases in detail, including their pre-conditions, main flow, alternative flows, and post-conditions.

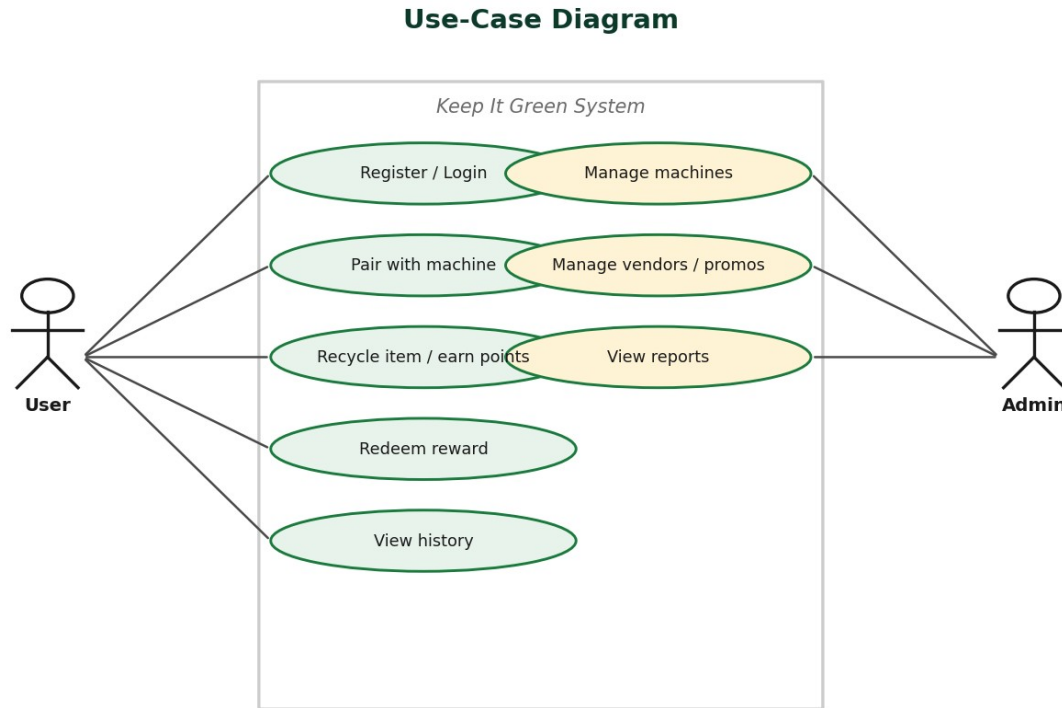


Figure 3.1: Use-case diagram of the Keep It Green system.

3.5.1 Use Case: Recycle an Item

Table 3.2: Use-case specification — Recycle an Item.

Field	Detail
Actors	End User, Machine, Backend
Pre-conditions	The user is registered and logged in; the machine is online and available.
Trigger	The user approaches the machine and starts a session.
Main flow	1. The machine requests a session; the backend returns a unique pairing code. 2. The machine displays the code as QR and digits. 3. The user pairs their phone to the code. 4. The backend binds the code to the user and confirms pairing. 5. The user inserts an item. 6. The machine detects and confirms the item (confidence ≥ 0.65 , stable ≥ 1.2 s). 7. The machine sorts the item and submits the detection with its key. 8. The backend credits the user and the app shows the updated points. 9. Steps 5–8 repeat for further items. 10. The user finishes the session; the machine returns to idle.
Alternative flows	6a. The item is not confidently recognized → the machine rejects it and prompts the user to try another item. 3a. The code has expired → the machine requests a new session.
Post-conditions	The user's balance and history reflect every accepted item; the session is closed.

3.5.2 Use Case: Redeem a Reward

Table 3.3: Use-case specification — Redeem a Reward.

Field	Detail
Actors	End Users, Backend
Pre-conditions	The user is logged in and has sufficient points.

Field	Detail
Main flow	1. The user opens the rewards catalogue. 2. The user selects a reward whose cost is within their balance. 3. The backend deducts the points, records a redemption, and reveals the promotional code. 4. The app displays the code and adds it to the user's history.
Alternative flows	2a. The reward costs more than the balance → the backend rejects the redemption and the app explains why.
Post-conditions	The user's balance is reduced by the reward's cost; a redemption record exists; the code is revealed only to that user.

3.5.3 Use Case: Manage Machines (Administrator)

Table 3.4: *Use-case specification — Manage Machines.*

Field	Detail
Actors	Administrator, Backend
Pre-conditions	The administrator is logged in with the administrator role.
Main flow	1. The administrator adds a machine with a name and location. 2. The administrator enables or disables the machine's availability. 3. The administrator views the machine's transaction report. 4. The administrator may delete the machine.
Alternative flows	A non-administrator attempting any of these actions is refused (HTTP 403).
Post-conditions	The machine registry reflects the administrator's changes.

3.5.4 Use Case: Pair with a Machine

Table 3.5: *Use-case specification — Pair with a Machine.*

Field	Detail
Actors	End User, Machine, Backend
Pre-conditions	The user is logged in; the machine displays an active, unpaired code.
Main flow	1. The user opens the Scan screen. 2. The user scans the QR code or types the six-digit code. 3. The app sends the code and the user's token to the backend. 4. The backend verifies the code is active and unpaired, binds it to the user, and confirms. 5. The app opens the live-session screen.
Alternative flows	4a. The code is expired or already paired → the backend refuses and the app asks the user to try again with a fresh code.
Post-conditions	The session is bound one-to-one to the user; subsequent detections credit this user.

3.5.5 Use Case: Finish a Session

Table 3.6: *Use-case specification — Finish a Session.*

Field	Detail
Actors	End User, Machine, Backend
Pre-conditions	The user has an active paired session.
Main flow	1. The user taps Finish in the app. 2. The app calls the end-session endpoint. 3. The backend marks the session stopped. 4. On its next status poll the machine observes the change and returns to the idle screen.
Alternative flows	The user simply walks away → the unpaired code eventually expires; a paired session can also be ended from the machine.

Field	Detail
Post-conditions	The session is closed; the machine is idle and ready for the next user.

3.6 User Stories

To keep the requirements grounded in real value, they were also expressed as user stories from the perspective of each actor:

- As a user, I want to earn points instantly when I recycle a bottle or a can, so that recycling feels worthwhile.
- As a user, I want to see my points go up live while I use the machine, so that I trust the machine counted my items.
- As a user, I want the promo code to stay hidden until I actually redeem it, so that codes are not leaked.
- As a user, I want to change reward codes for real vendor offers, so that my recycling has tangible benefit.
- As an administrator, I want to add machines and rewards and see each machine's activity, so that I can run the network.
- As an operator, I want the machine to reject anything that is not a bottle or a can, so that the bins stay clean and the rewards stay honest.

3.7 Constraints and Assumptions

The prototype is developed under a number of explicit constraints and assumptions. It is assumed that a single item fills the machine's camera view during insertion, which justifies treating detection as a single-object problem at inference time. It is assumed that the machine's computer has network access to reach the backend, though brief interruptions are considered in future work. The reward classes are constrained to plastic bottles and aluminium cans. The physical prototype is a supervised demonstration unit rather than a hardened, weather-sealed product. These constraints are reasonable for a graduation prototype and are revisited critically in Chapters 12 and 13.

3.8 Feasibility Study

The feasibility of the project was assessed along four dimensions. Technically, the project is feasible: object detection on commodity hardware is a mature capability, the .NET and Flutter platforms are well-supported, and the team possessed or acquired the necessary skills; the principal technical risk, real-time accuracy, was mitigated by careful model selection and

evaluation. Economically, the project is feasible and attractive: it is built on free and open-source software and low-cost commodity hardware, and it is deployed on free hosting, so the marginal cost of the prototype is dominated by the inexpensive hardware; the business model, in which partner vendors fund rewards in exchange for footfall and brand exposure, is examined in Chapter 12. Operationally, the system is feasible because it fits existing behaviour — people already carry smartphones — and requires no legislative change. Legally and environmentally, the system aligns with national policy goals for higher recycling rates and introduces no regulatory obstacles. The study concluded that the project was feasible on all four dimensions.

3.9 Risk Analysis

The principal risks to the project were identified early and managed throughout. Table 3.2 records the most significant risks, their assessed likelihood and impact, and the mitigation adopted for each.

Table 3.7: *Risk register (excerpt).*

Risk	Likelihood	Impact	Mitigation
Detector confuses/accepts non-target items	Medium	High	Confidence-threshold rejection; stability buffer; test on unseen data
Class imbalance harms aluminium accuracy	Medium	Medium	Heavy augmentation; per-class metrics; monitor recall
Wrong user credited on a shared machine	Low	High	ATM session model; unique codes; isolation tests
Camera fails to open on the phone	Medium	Medium	Diagnostic UI; bundled ML Kit; manual-code fallback
Public deployment exposes secrets	Medium	High	Config-based secrets; secret rotation; HTTPS
Network outage at the machine	Low	Medium	Documented; offline queue proposed as future work

3.10 Summary of the Analysis

The analysis in this chapter establishes a clear and testable picture of what the system must do and how well it must do it. The functional requirements define the recycling, reward, and administrative behaviours; the non-functional requirements fix the demanding quality targets of real-time response, high detection accuracy, strict session correctness, and strong security; the use-case specifications describe the principal interactions in enough detail to design against; and the feasibility study and risk register confirm that the project is achievable and that its principal risks can be managed. These requirements are the yardstick against which the design of Chapter 4, the implementation of the chapters that follow, and the testing of Chapter 10 are

all measured, and the traceability of Section 10.10 closes the loop by showing that each is satisfied.

Chapter 4 System Architecture and Design

This chapter describes how the system is structured to meet the requirements of Chapter 3. It presents the overall architecture and the architectural style, the layered design of the backend, the pairing and session model that is central to the whole system, the end-to-end data flow of a recycling session, the database design, the design of the web API, the security architecture, the design patterns employed, and the principles governing the user interface and localization.

4.1 Architectural Overview

Keep It Green is composed of five cooperating components, listed with their responsibilities in Table 4.1. The components communicate exclusively through the backend's web API over HTTPS, exchanging JSON. The overriding architectural principle, introduced in Chapter 1 and enforced everywhere, is that no component ever accesses the database directly: the machine, the phone, and the browser are all clients of the same authenticated API. This concentrates business logic and security in a single, testable place and makes every other component thinner and safer.

Table 4.1: *System components and their responsibilities.*

Component	Technology	Responsibility
AI Machine	Python, YOLOv8, Arduino	Detect items, sort them, submit transactions
Backend API	.NET 10, EF Core, SQL Server	Users, sessions, points, rewards, authentication
Mobile App	Flutter / Dart	Pair, view live session, redeem rewards
Web App	Vanilla JavaScript, PWA	User and administrator dashboard in the browser
Kiosk UI	HTML served by the backend	On-machine ATM-style screen

4.2 Architectural Style

The system follows a client–server architecture organized around a RESTful web API. The server is stateless with respect to client sessions in the HTTP sense — each request carries its own authentication — while persistent state lives in the database. This style was chosen because it decouples the many heterogeneous clients (a Python machine, a Flutter app, a JavaScript browser) from the server, allowing each to be developed and deployed independently as long as it respects the API contract. Internally, the server itself is organized as a layered architecture, described next, which separates the concerns of handling HTTP, executing business logic, and accessing data.

4.3 Backend Layered Architecture

The backend is divided into four layers, shown in Figure 4.1. Requests enter at the Controllers layer, which is responsible only for HTTP concerns: routing, deserializing request bodies, applying authentication and authorization, and shaping responses. Controllers delegate to the Services layer, which contains the business logic — starting sessions, generating unique codes, awarding points, resolving redemptions — and knows nothing about HTTP. Services use the Repositories layer, defined behind interfaces, which encapsulates data access and hides the details of the database. Repositories operate on the Entity Framework Core DbContext and the entity classes, which map to the database tables. Each layer depends only on the layer beneath it, and dependencies are injected rather than hard-wired, so that any layer can be tested in isolation.

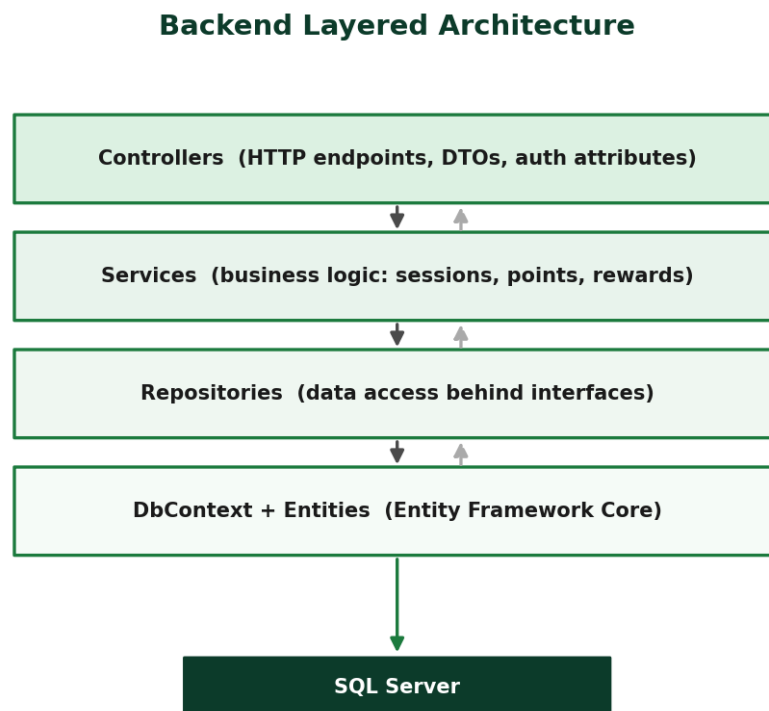


Figure 4.1: The four-layer architecture of the backend.

4.4 The Pairing and Session Model

The single most important design decision in the system is how a physical machine, shared by many people, safely attributes each recycled item to the correct user. The answer is an ATM-style pairing and session model. When a user approaches, the machine asks the backend to start a session; the backend generates a six-digit pairing code that is guaranteed not to collide with any currently active code, stores it as a pending session, and returns it; the machine displays the code as a QR code and as digits. The user's phone then pairs to that code, and the backend

binds the code to that user, promoting the session to active. From that moment, every detection the machine submits carries the code, and the backend credits the item to exactly the one user bound to it. When the user finishes, the backend marks the session inactive; the machine, which polls the session status, observes this and returns to idle.

This model has three safety properties, each verified by the isolation tests of Chapter 10: pairing codes are globally unique at any instant, so two concurrent users can never share a code; each code maps to exactly one user, so an item is never ambiguous; and a code that is already paired cannot be re-paired by a second user, so a session cannot be hijacked. Figure 4.2 shows the full sequence of a session.

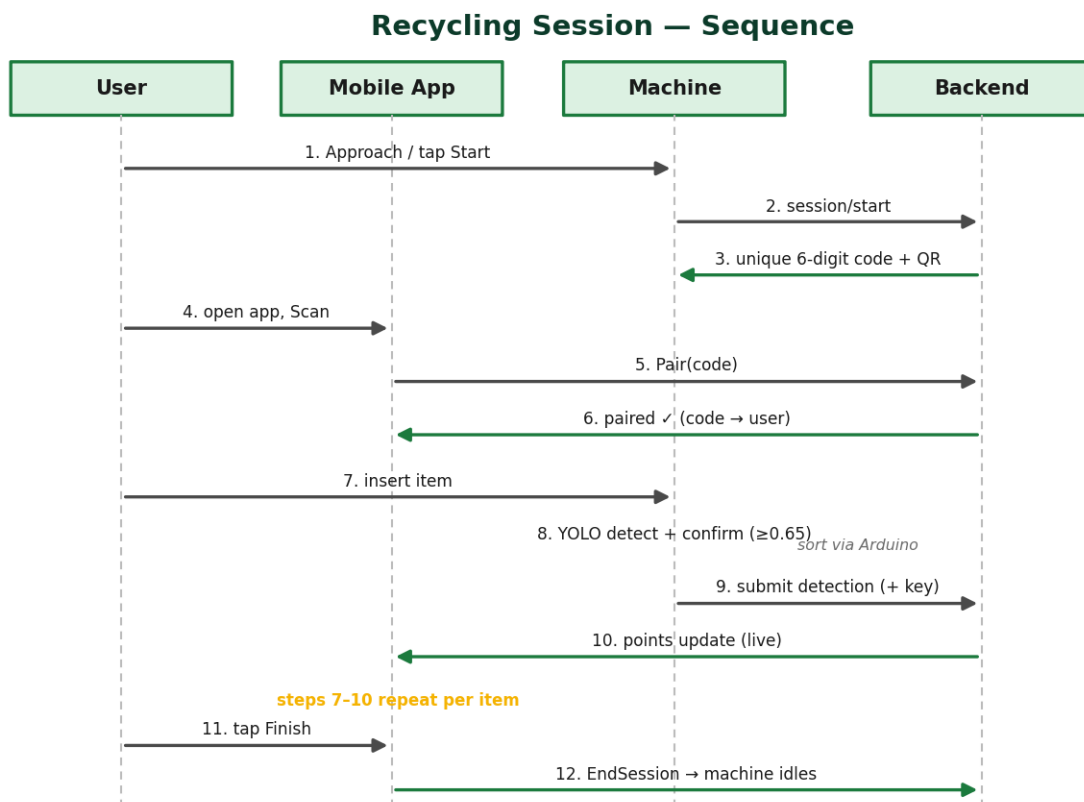


Figure 4.2: Sequence of a complete recycling session.

4.5 End-to-End Data Flow

Tracing a single item through the system illustrates how the components cooperate. The user taps to start; the machine calls the session-start endpoint; the backend creates a pending Otp (session) record with a unique code and returns it; the machine renders the code. The user opens the app and pairs; the app calls the pairing endpoint with the code and the user's token; the

backend sets the Otp's user and status. The user inserts a bottle; the machine's camera captures frames; the YOLO model detects a plastic bottle with confidence above the threshold; after the detection is stable for the required interval, the machine sends the byte that opens the plastic compartment and calls the detection-submit endpoint with the code, the item type, and its machine key. The backend validates the key, resolves the code to the user, creates a Transaction of ten points, and increases the user's balance; the app, polling the session, shows the new total. When the user finishes, the app calls the end-session endpoint, the Otp is marked stopped, and the machine's next status poll returns it to idle. Every step that changes data is authenticated, and every step passes through the API.

4.6 Database Design

The persistent state of the system is held in a relational SQL Server database, accessed through Entity Framework Core. The schema is deliberately compact, organized around a small set of entities whose relationships are shown in Figure 4.3 and detailed in the tables that follow.

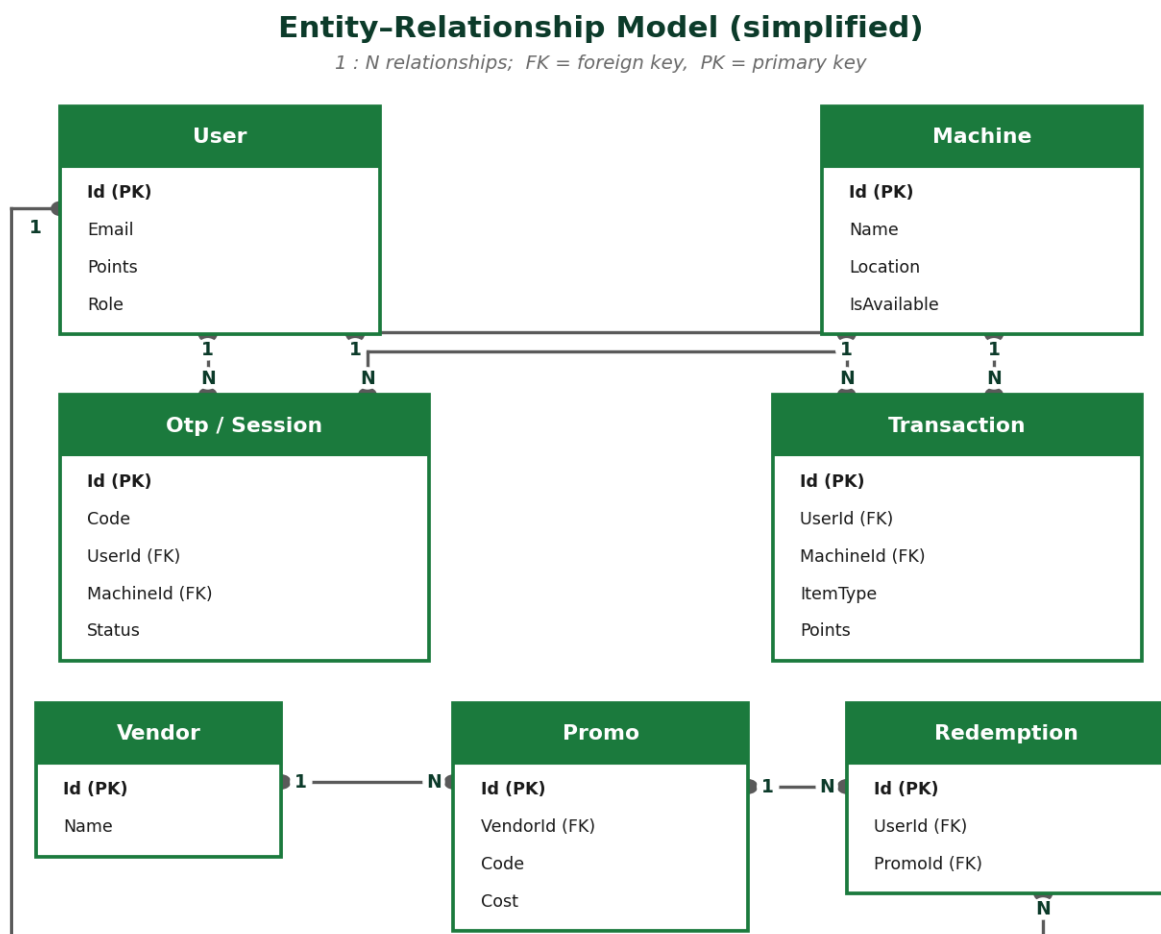


Figure 4.3: Entity-relationship model of the database.

4.6.1 Core Entities

The User entity, managed by ASP.NET Identity, holds the account credentials, the point balance, and the role. The Machine entity describes a physical machine. The Otp entity represents a pairing code and the session it governs, linking a user to a machine for the session's duration. The Transaction entity records each accepted item. The Vendor, Promo, and Redemption entities implement the rewards catalogue and its redemptions. Tables 4.2 and 4.3 detail the two most central entities.

Table 4.2: *The Otp (session) entity.*

Column	Type	Purpose
Id	int (PK)	Primary key
Code	string(6)	The pairing code shown by the machine
UserId	FK → User	The paired user (null until paired)
MachineId	FK → Machine	The machine that issued the code
Status	enum	Pending, Active, or Stopped
ExpiresAt	datetime (UTC)	When the unpaired code lapses

Table 4.3: *The Transaction entity.*

Column	Type	Purpose
Id	int (PK)	Primary key
UserId	FK → User	The credited user
MachineId	FK → Machine	The machine that accepted the item
ItemType	enum	plastic_bottle or aluminum_can
Points	int	Points awarded (10 or 15)
CreatedAt	datetime (UTC)	When the item was accepted

The remaining core entities are detailed in Tables 4.4 to 4.6. The User entity holds the account and the authoritative point balance; the Machine entity describes a physical unit; and the rewards entities — Vendor, Promo, and Redemption — implement the catalogue and its redemptions.

Table 4.4: *The User entity.*

Column	Type	Purpose
Id	string (PK)	Identity primary key
Email / UserName	string	Login identity
PasswordHash	string	Salted password hash (ASP.NET Identity)
Points	int	Authoritative point balance
Role	string	User or Admin

Table 4.5: *The Machine entity.*

Column	Type	Purpose
Id	int (PK)	Primary key
Name	string	Human-readable machine name
Location	string	Where the machine is installed
IsAvailable	bool	Whether the machine accepts sessions

Table 4.6: *The rewards entities (Vendor, Promo, Redemption).*

Entity	Key columns	Purpose
Vendor	Id, Name	A partner offering rewards
Promo	Id, VendorId (FK), Code, Cost	A redeemable offer with a point cost
Redemption	Id, UserId (FK), PromoId (FK), CreatedAt	A record that a user spent points on a promo

4.6.2 Relationships and Integrity

The relationships are enforced by foreign keys and by Entity Framework Core migrations, which are applied automatically when the backend starts. A user has many sessions, transactions, and redemptions; a machine has many sessions and transactions; a vendor has many promos; and a promo has many redemptions. Referential integrity guarantees that no transaction can reference a non-existent user or machine, and indexes on the foreign keys and on the pairing code keep the common queries — resolving a code, listing a user's history, reporting a machine's transactions — efficient.

4.6.3 Normalization

The schema is normalized to third normal form: every non-key attribute depends on the key, the whole key, and nothing but the key. Reward points, for example, are not duplicated across tables; the user's balance is maintained as a single authoritative value and the transactions provide the auditable history from which it is derived. This avoids update anomalies and keeps the data consistent, while the indexes ensure that normalization does not cost performance on the queries that matter.

4.7 API Design

The web API follows REST conventions. Resources — users, machines, sessions, transactions, vendors, promos, redemptions — are addressed by URL, and the HTTP verbs express intent: GET to read, POST to create, PUT to update, and DELETE to remove. Requests and responses carry JSON. Standard status codes communicate outcomes: 200 for success, 400 for a malformed request, 401 for missing or invalid authentication, 403 for an authenticated user lacking the required role, and 404 for a missing resource. The API is described by an OpenAPI

(Swagger) document that is served live, allowing the contract to be explored and exercised directly. The complete endpoint reference appears in Appendix A.

As an example, a client logs in by posting credentials and receives a signed token, which it then presents on subsequent requests. Listing 4.1 shows this exchange.

Listing 4.1: *Login request and token response (illustrative).*

```
POST /api/User/Login
Content-Type: application/json
{ "email": "user@example.com", "password": "....." }

HTTP/1.1 200 OK
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "User", "coins": 0 }
```

4.8 Security Architecture

Two distinct authentication mechanisms protect the system, matching the two kinds of client. Human users authenticate with JSON Web Tokens: on login the backend issues a token, signed with a secret key, that carries the user's identity and role and expires after a set interval; every subsequent request presents this token, which the backend validates without a database lookup. The machine authenticates differently, with a secret key sent in an HTTP header (X-Machine-Key) that only genuine machines possess; a detection submission without the correct key is rejected. Authorization is role-based: administrative endpoints require the administrator role, and user-flow endpoints require the user role, so the two capability sets are cleanly separated. Because the deployed backend runs behind the host's HTTPS-terminating reverse proxy, it is configured to honour forwarded headers so that it correctly recognizes secure connections. A brief threat model — impersonation, replay, code guessing, secret exposure — and the corresponding mitigations are discussed in Chapter 6.

4.9 Design Patterns

Several established design patterns structure the implementation. The Repository pattern hides data access behind interfaces, decoupling the services from Entity Framework. Dependency Injection, built into ASP.NET Core, supplies each component with its collaborators and makes testing straightforward. The Data Transfer Object pattern separates the shapes exchanged over the API from the internal entity classes, so that the database schema can evolve without breaking clients and so that sensitive fields are never inadvertently exposed. The layered architecture itself is an application of separation of concerns. On the clients, the mobile and web applications concentrate all network access in a single service module, an application of the same principle.

4.10 User-Interface and Experience Design

The user-facing surfaces share a single brand and a set of design principles. The visual identity uses a forest-and-spring green palette with a sunlit accent, and a distinctive display typeface, to convey an environmental, optimistic character consistently across the machine, the app, and the web. The machine's kiosk screen is designed to be understood without instruction, in the manner of an automated teller machine: large targets, one decision at a time, and immediate feedback. The mobile application follows Material 3 conventions so that it feels native to Android users. Accessibility considerations — legible contrast, large touch targets, and clear feedback — inform all three surfaces. The detailed screen designs are presented in Chapters 7 and 8.

4.11 Localization and Time

The system serves a specific locale, and time is handled carefully to reflect this. All timestamps are stored in Coordinated Universal Time in the database, which avoids ambiguity and daylight-saving errors at the point of storage, and they are converted to Cairo local time (the Africa/Cairo zone, with correct daylight-saving handling) at the point of display on every client. As a result, users and administrators always see times that match their own context, regardless of where the server runs.

4.12 Scalability and Performance Considerations

Although the prototype operates at modest scale, the architecture was chosen so that it could grow. The stateless, token-based API means that additional application instances can be added behind a load balancer without shared session state to coordinate, so the service tier scales horizontally. The database is the natural point of contention at large scale, and it is kept efficient by indexing the columns on which the hot queries depend — resolving a pairing code, listing a user's transactions, and reporting a machine's activity — so that these operations remain fast as the tables grow. The machine's real-time work is performed locally on its own computer and places only a light, intermittent load on the backend when it submits a detection, so adding more machines increases backend load only gradually and predictably.

Performance was a first-class design concern because of the one-second real-time requirement. That budget is met on the machine side by the efficient detector and the local inference described in Chapter 5, and on the backend side by keeping the per-request work small: authentication is a signature check rather than a database lookup, and the common operations

touch few, well-indexed rows. The result is a system whose responsiveness does not depend on scale, which is exactly the property a network of machines serving many users requires.

Chapter 5 The Artificial Intelligence Module

The artificial-intelligence module is the component that gives the machine its intelligence: the ability to look at an inserted item and decide, in real time, whether it is a plastic bottle, an aluminium can, or neither. This chapter describes that module in depth. It explains how the recognition problem was framed and why non-recyclable items are rejected by confidence rather than by a dedicated class; how the training data were collected from four sources and unified; what the data look like; how they were augmented; the architecture of the chosen model and why it was chosen; how the model was trained, including the loss functions and hyperparameters; how it was evaluated and the results it achieved; and how it was exported and integrated into the machine's real-time control loop. The results reported here are drawn from the project's training notebook and reflect the model actually deployed.

5.1 Problem Framing and Open-Set Rejection

The machine must do two things at once: recognize the two rewarded materials, and reject everything else. The second requirement is subtler than the first. A recycling machine in the real world will be shown not only bottles and cans but also hands, phones, keys, food wrappers, and countless other objects, and it must not reward them. The obvious approach is to add a third class, “other”, and train the model to recognize it. This approach was rejected for a concrete, data-driven reason: the raw material available was overwhelmingly dominated by non-target objects, with roughly forty-four thousand “other” images against about four thousand plastic and one thousand aluminium. A class that large would dominate the training loss, bias the model toward predicting “other”, and squander model capacity on objects the machine never rewards, while still failing to cover the truly open-ended space of possible non-target items.

Instead, the module adopts open-set rejection by confidence thresholding. The model is trained on only the two rewarded classes, and the detector's own confidence score is used as the gate. At inference, if no detection reaches a confidence of 0.65, the item is treated as non-recyclable and rejected; only a confident detection of a bottle or a can is accepted and rewarded. This reframes an impossible closed-set problem (enumerate everything) as a tractable open-set one (recognize two things well, and distrust everything uncertain). The value of the 0.65 threshold is justified empirically in Section 5.15. Table 5.1 states the class scheme and its consequences.

Table 5.1: *Detection classes, rewards, and actions.*

Class	Reward	Action
plastic_bottle	+10 points	Sort into the plastic compartment
aluminum_can	+15 points	Sort into the aluminium compartment
(confidence < 0.65)	none	Reject; prompt the user to try another item

5.2 Data Collection: Four Datasets

A detector is only as good as the data it learns from, and robustness in the field demands diversity in lighting, angle, background, and the condition of the item — intact or crushed, labelled or bare. To achieve this diversity, four publicly available object-detection datasets were combined — every one of them supplying real, human-drawn bounding boxes, a deliberate change from earlier iterations that synthesized full-frame boxes from classification images and, as a result, localized poorly. The Drinking Waste Classification set contributes clean, canonical bottles and cans; three Roboflow community datasets — Beverage Containers, YOLO Waste Detection, and Can/Bottle/Pack — contribute tightly-drawn boxes across cluttered, realistic scenes. From each source only genuine plastic bottles and metal drink cans were kept, using a verified per-dataset class map audited from that dataset's real class list, so that a glass bottle, a bottle cap, a cup, or a tetra-pak is discarded rather than mislabelled. Table 5.2 summarizes the contribution of each source.

Table 5.2: *Composition of the unified dataset by source.*

Source	Type	Contribution
Drinking Waste Classification	Real boxes	2,259 clean bottles/cans (PET, AluCan)
Beverage Containers (Roboflow)	Real boxes	3,358 images — bottle-plastic, tin can
YOLO Waste Detection (Roboflow)	Real boxes	4,563 images — plastic bottle, aluminum can
Can, Bottle & Pack (Roboflow)	Real boxes	1,717 images — plasticBottle_*, can_*

5.3 Data Unification Pipeline

Merging four datasets with different formats, label conventions, and category sets into a single, consistent detection dataset required a deliberate pipeline. Each source was parsed according to its own format, and — critically — every original bounding box was kept exactly as annotated; no box was ever generated or moved, since synthetic boxes had degraded localization in earlier versions. Only the class identifier was remapped, through a verified per-dataset name map, to the project's two-class scheme. A fifth candidate, the TACO litter dataset, was automatically rejected because the available export carried a single generic class with polygon labels, unusable for two-class bounding-box detection — an illustration of the pipeline

refusing to guess. Across all sources, only the plastic and aluminium categories were kept, and every image was written out in the unified label format expected by the training framework, together with a dataset descriptor naming the two classes. The result is a single coherent dataset that the model can be trained on directly. Figure 5.1 depicts the pipeline end to end.

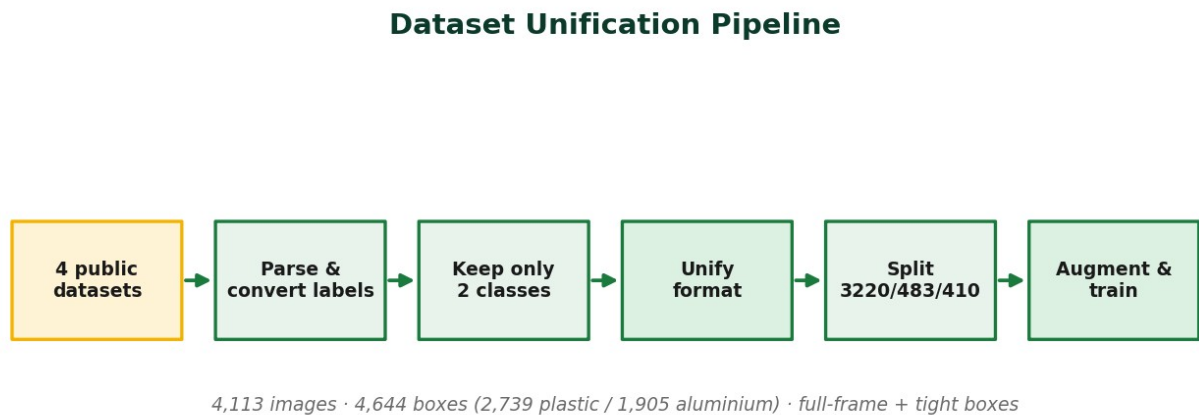


Figure 5.1: *The dataset unification pipeline, from four raw sources to an augmented training set.*

Two preprocessing details are worth noting. First, images were standardized to a common size for training (letterbox-resized so that the aspect ratio is preserved and no object is distorted), which is necessary because the four sources differ in resolution. Second, the label files were validated programmatically after conversion — every box was checked to lie within the image bounds and to carry one of the two permitted class identifiers — so that a formatting error in one source could not silently corrupt training. Only after this validation passed was the dataset used.

5.4 Dataset Statistics and Splits

The unified dataset contains 11,897 images holding 16,772 labelled bounding boxes — 8,534 for plastic and 8,238 for aluminium, a naturally balanced split. It is partitioned into three disjoint subsets: 10,327 images for training, 978 for validation, and 592 for testing. The training set is what the model learns from; the validation set is used during training to monitor generalization and to select the best epoch; and the test set is held back entirely and used only once, at the end, to give an unbiased estimate of real-world performance. Figure 5.2 shows the class distribution, the split sizes, and the provenance of the data.

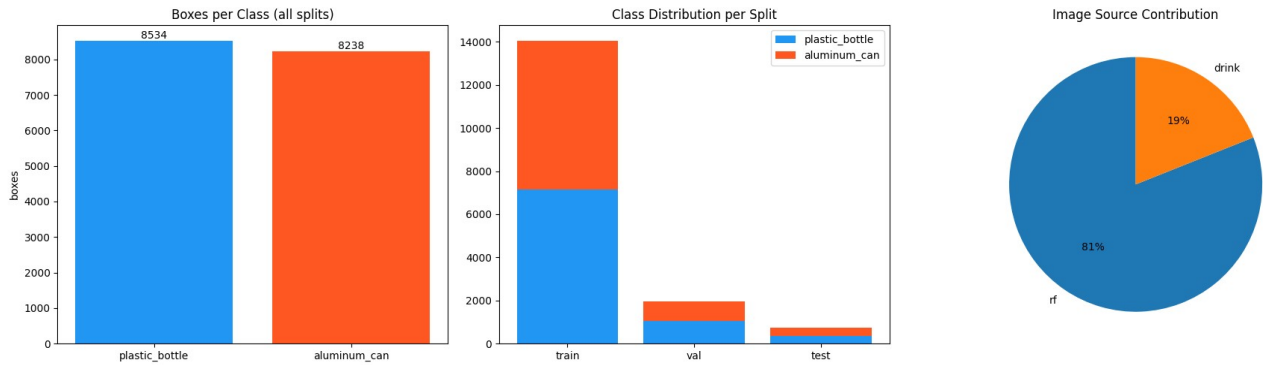


Figure 5.2: Class distribution, split sizes, and data provenance of the unified dataset.

The two classes are well balanced — 8,534 plastic boxes against 8,238 aluminium, a ratio near 1.04 — because the source datasets were chosen and subsampled to avoid the imbalance that hampered earlier versions. This balance is one reason the per-class results reported in Section 5.12 are trustworthy, and it removes any need to lean on augmentation merely to compensate for a starved class.

5.5 Exploratory Data Analysis

Before training, the data were analysed to understand their characteristics and to inform design decisions. Three properties proved important. First, the distribution of source image resolutions, shown in Figure 5.3, has a median around 481 by 640 pixels; this informed the decision to train at an image size of 640 pixels, which is large enough to preserve detail without wastefully upscaling the smaller images. Second, the geometry of the bounding boxes, shown in Figure 5.4, confirms that the labels are genuinely localized: only about one per cent of boxes are near-full-frame, so the model learns not merely what an item looks like but precisely where it sits in the frame — the single most important improvement over earlier versions, whose synthetic full-frame boxes had taught it to classify the whole image rather than locate the object. Third, the correlation of label and spatial statistics, shown in Figure 5.5, was examined to confirm that no spurious artefact (such as a class always appearing in one corner) could let the model cheat.

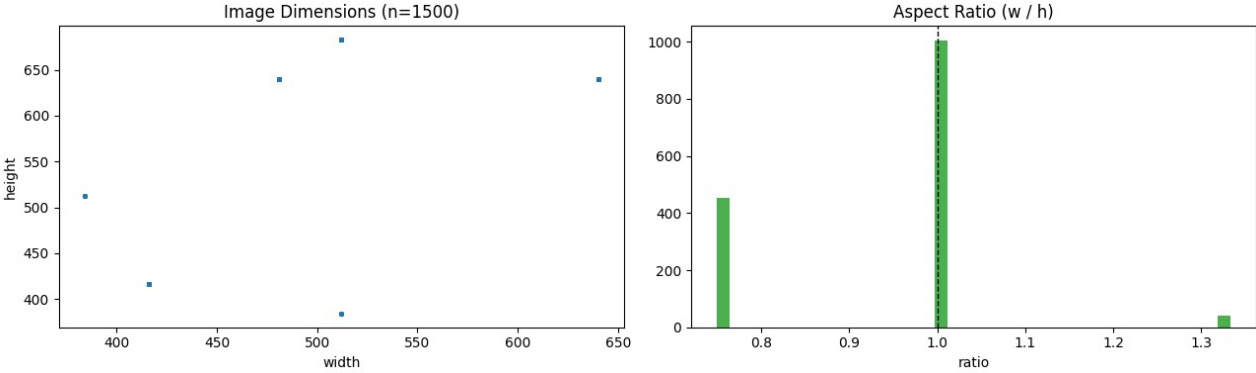


Figure 5.3: *Distribution of source image resolutions; the median is about 481×640 pixels.*

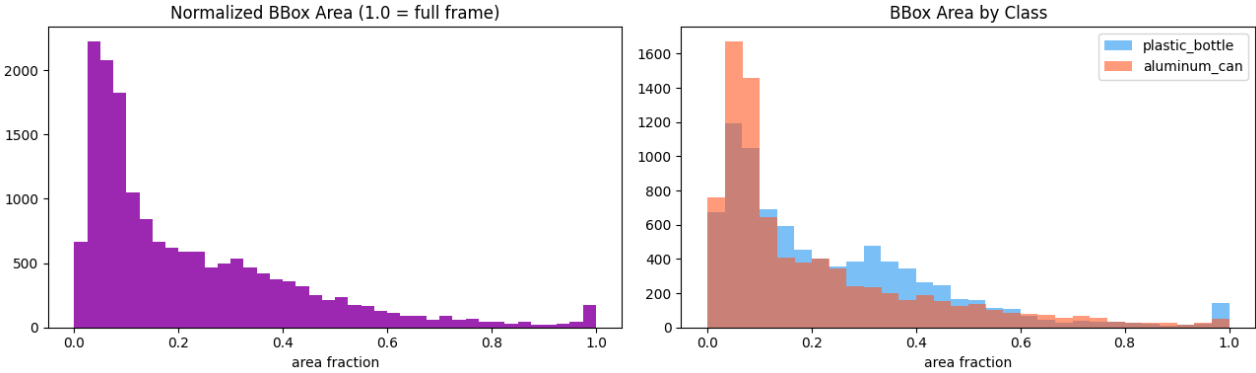


Figure 5.4: *Bounding-box geometry: the boxes are real, human-drawn annotations — only about 1% are near-full-frame.*

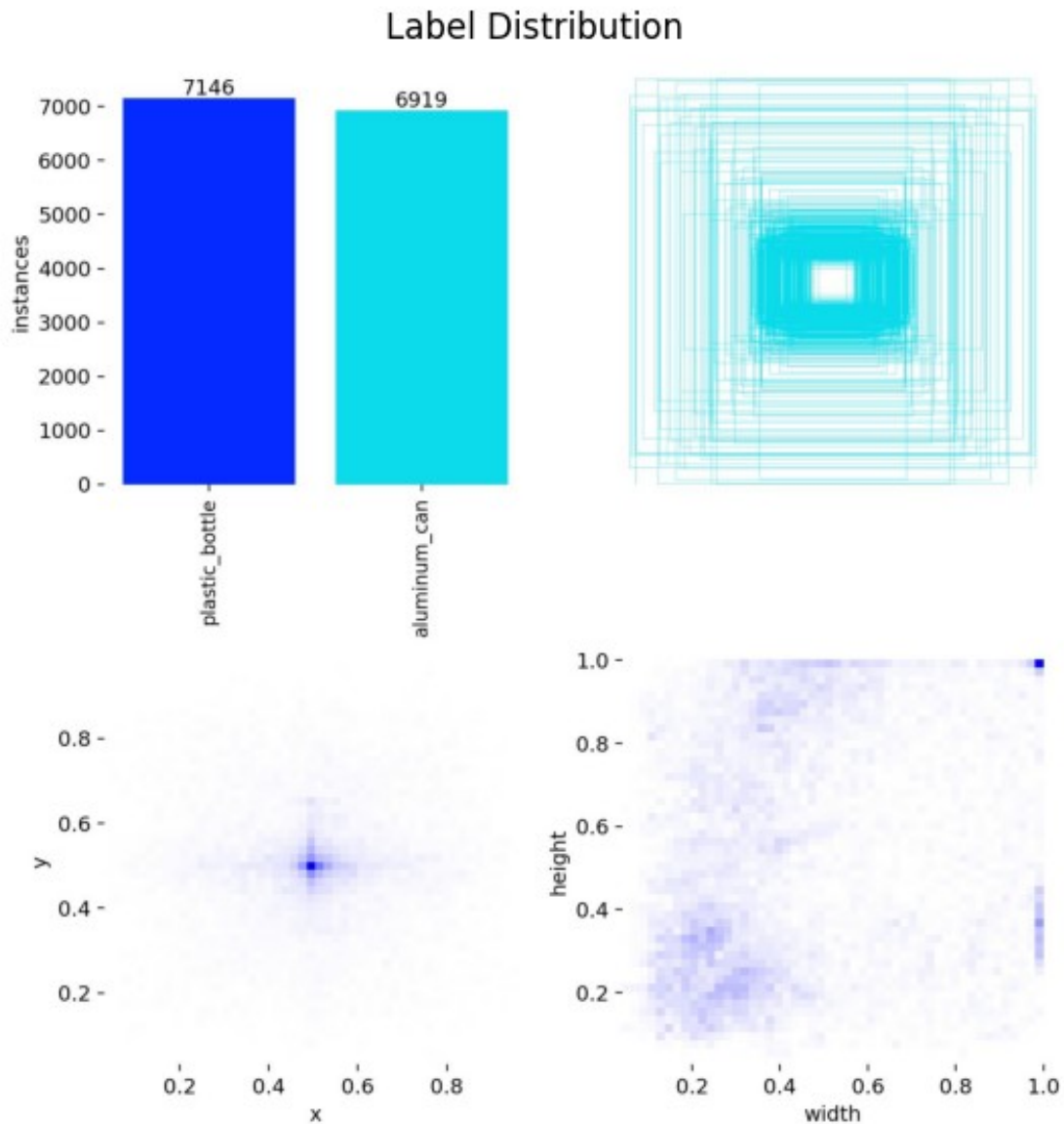


Figure 5.5: Correlogram of label and box statistics, used to check for spurious spatial biases.

Finally, a direct visual inspection of training images with their ground-truth boxes, shown in Figure 5.6, served as a sanity check that labels were correctly aligned with objects across all four sources before a single epoch of training was run.



Figure 5.6: Sample training images overlaid with their ground-truth bounding boxes.

5.6 Data Augmentation

To compensate for the class imbalance and, more importantly, to make the model robust to the variability it will meet in the field, an aggressive data-augmentation strategy was applied during training. Each technique addresses a specific kind of variation. Mosaic augmentation stitches four training images into one, exposing the model to many objects, scales, and contexts in a single training step and greatly improving robustness to background and scale. Mixup blends two images and their labels, which regularizes the model and softens its decision boundaries. HSV colour jitter perturbs hue, saturation, and brightness, which is essential for a machine that may operate under different lighting. Geometric transforms — rotation, translation, scaling, and horizontal flipping — teach invariance to the item's position and orientation, since a user may insert a bottle any way up. Together these augmentations multiply the effective size and diversity of the training set far beyond its 10,327 raw training images.

5.7 Model Architecture: YOLOv8s

The chosen model is YOLOv8s, the “small” variant of the YOLOv8 detector, comprising approximately 11.1 million parameters. Its architecture has three parts. The backbone, a CSPDarknet network, extracts a hierarchy of visual features from the input image, from low-level edges and textures in its early layers to high-level object parts in its deeper layers; the cross-stage-partial design splits and recombines feature channels to improve gradient flow and efficiency. The neck, a path-aggregation feature-pyramid network, fuses features across scales so that both small and large objects are represented well, combining semantically strong coarse features with spatially precise fine ones. The head is decoupled and anchor-free: it predicts, at

each spatial location and scale, the object's class and the geometry of its bounding box directly, without the predefined anchor boxes that earlier YOLO versions relied on, which simplifies training and improves accuracy on objects of unusual shape.

Among the YOLOv8 size variants, the small model was selected deliberately. The nano variant, while faster, risks under-fitting the harder, less-represented aluminium class; the medium and larger variants offer marginally higher accuracy but are unnecessarily heavy for a two-class problem and would slow inference on a laptop-class CPU. The small variant sits at the balance point that satisfies both the accuracy target and the real-time requirement, as the results below confirm. Table 5.3 sets out the trade-off across the variants that informed this choice.

Table 5.3: *YOLOv8 size variants and the selection rationale.*

Variant	Params	Character	Fit for this project
YOLOv8n (nano)	~3.2 M	Fastest, least accurate	Risk of under-fitting aluminium
YOLOv8s (small)	~11.1 M	Balanced	Chosen — accurate and real-time on CPU
YOLOv8m (medium)	~25.9 M	More accurate, heavier	Unnecessary for two classes
YOLOv8l / x	~43–68 M	Highest accuracy, slow	Too heavy for the deployment target

5.8 Transfer Learning

Rather than training from random initialization, the model was initialized with weights pre-trained on the large, general-purpose COCO object-detection dataset, and then fine-tuned on the recycling dataset — the technique known as transfer learning. Because the pre-trained network already understands generic visual concepts such as edges, contours, textures, and common object shapes, fine-tuning needed only to specialize this knowledge to the two target materials. Transfer learning dramatically reduces the amount of data and training time required to reach high accuracy and is standard practice in applied computer vision; it is a large part of why a dataset of a few thousand images suffices to train an accurate detector here.

5.9 Training Configuration and Loss Functions

The model was trained on cloud infrastructure providing two NVIDIA Tesla T4 GPUs, which allowed a large batch size and short training times. The optimizer was AdamW, a variant of Adam with decoupled weight decay that generalizes well. Training ran for up to 100 epochs with early stopping: if the validation metric did not improve for 20 consecutive epochs, training halted and the best epoch's weights were retained, which prevents over-fitting and wasted computation. The full hyperparameter configuration is given in Table 5.3.

Table 5.4: *Training hyperparameters.*

Hyperparameter	Value	Rationale
Base model	YOLOv8s (COCO-pretrained)	Balance of accuracy and speed; transfer learning
Epochs	100 (early stop, patience 20)	Enough to converge; stop to avoid over-fitting
Optimizer	AdamW	Stable convergence, good generalization
Learning rate	0.001 (initial)	Standard fine-tuning rate
Batch size	32	Large batch enabled by dual T4 GPUs
Image size	640×640	Matches median source resolution, preserves detail
Hardware	$2 \times$ NVIDIA Tesla T4	Parallel training, short wall-clock time
Augmentation	Mosaic, mixup, HSV, geometric	Robustness to lighting, scale, orientation

The training objective combines three loss terms, as is standard for YOLOv8. A classification loss (binary cross-entropy) penalizes wrong class predictions. A localization loss based on Complete IoU (CIoU) penalizes boxes that overlap the ground truth poorly, accounting not only for overlap but also for the distance between box centres and the similarity of their aspect ratios. A distribution-focal loss (DFL) refines the precise position of the box edges by treating each coordinate as a distribution rather than a single value. The total loss is a weighted sum of the three, and it is this quantity that the optimizer minimizes:

$$L = \lambda_{box} \cdot L_{CIoU} + \lambda_{cls} \cdot L_{BCE} + \lambda_{dfl} \cdot L_{DFL}$$

5.10 Training Process

Figure 5.7 shows the evolution of the losses and metrics over the course of training. The three loss curves — box, classification, and distribution-focal — decrease steadily on both the training and validation sets, and the validation metrics (precision, recall, and mean Average Precision) rise correspondingly and then plateau. Crucially, the training and validation curves track each other closely rather than diverging, which is the signature of healthy learning without over-fitting: the model is generalizing to unseen validation data, not merely memorizing the training set. Early stopping retained the best epoch on the plateau.

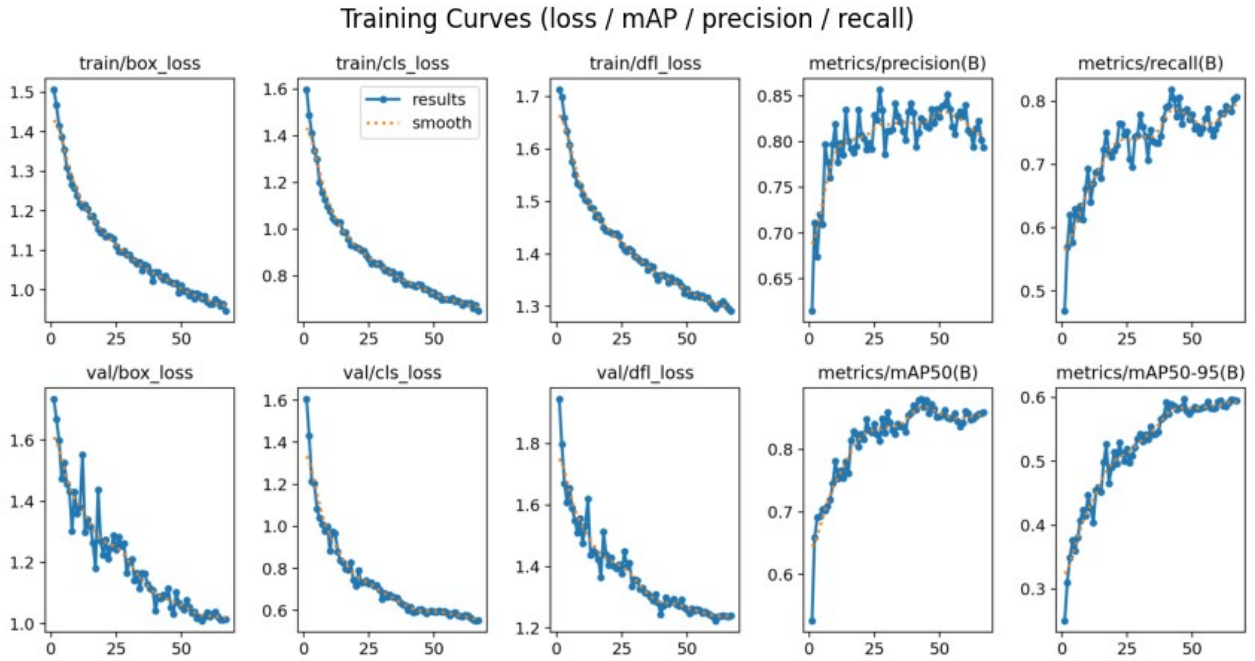


Figure 5.7: Training and validation loss and metric curves over training.

5.11 Evaluation Methodology

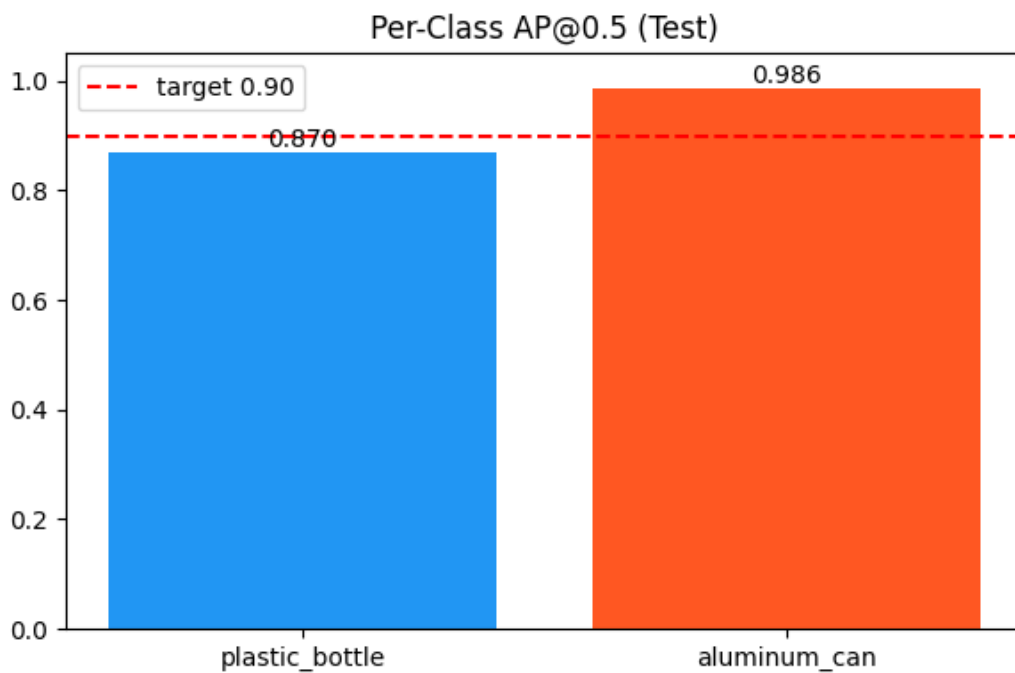
The final model was evaluated on the 592-image test set, which was isolated from the beginning and used neither for learning nor for epoch selection, so that its metrics estimate genuine field performance rather than performance on data the model has effectively seen. The metrics computed are those defined in Chapter 2: precision, recall, per-class Average Precision at an IoU threshold of 0.5, the mean of these (mAP@0.5), and the stricter mAP@0.5:0.95 averaged over IoU thresholds from 0.5 to 0.95. The project's success criterion, fixed in advance, was mAP@0.5 greater than 0.90.

5.12 Quantitative Results

The model comfortably exceeds its target. On the held-out test set it achieves a mAP@0.5 of 0.928, above the 0.90 criterion, together with a precision of 0.924 and a recall of 0.864; the stricter mAP@0.5:0.95 is 0.687, indicating that the model not only detects the items but localizes them precisely. The full results are given in Table 5.4. A particularly telling result is the per-class breakdown, visualized in Figure 5.8: aluminium cans are recognized almost perfectly, at an Average Precision of 0.986, while plastic bottles — far more visually varied, being clear, coloured, crushed, or heavily labelled — are the harder class at 0.870. This reverses the weakness of earlier versions, in which aluminium was the fragile class; here it is the strongest, and plastic recall is the single figure left to lift with more data.

Table 5.5: *Test-set detection metrics (held-out data).*

Metric	Value	Notes
mAP@0.5	0.928	Target > 0.90 — PASS
mAP@0.5:0.95	0.687	Stricter, averaged over IoU thresholds
Precision	0.924	Few false accepts
Recall	0.864	Few missed items
AP@0.5 — plastic_bottle	0.870	Per-class
AP@0.5 — aluminum_can	0.986	Per-class; exceeds plastic despite less data

**Figure 5.8:** *Per-class Average Precision at IoU 0.5; aluminium clears the 0.90 target and plastic sits just below at 0.87.*

5.13 Confusion-Matrix Analysis

The normalized confusion matrix, Figure 5.9, shows where the model's predictions land relative to the truth. The diagonal dominates and the off-diagonal confusion between the two materials is negligible — the model essentially never mistakes a bottle for a can or vice versa. Almost all of the residual error is with the background class in the form of missed detections, and it is concentrated in the harder plastic class, a portion of whose bottles are predicted as background on a given frame; aluminium is recognized nearly perfectly. These misses are exactly what the inference-time confidence gate and the multi-frame stability buffer are designed to tolerate.

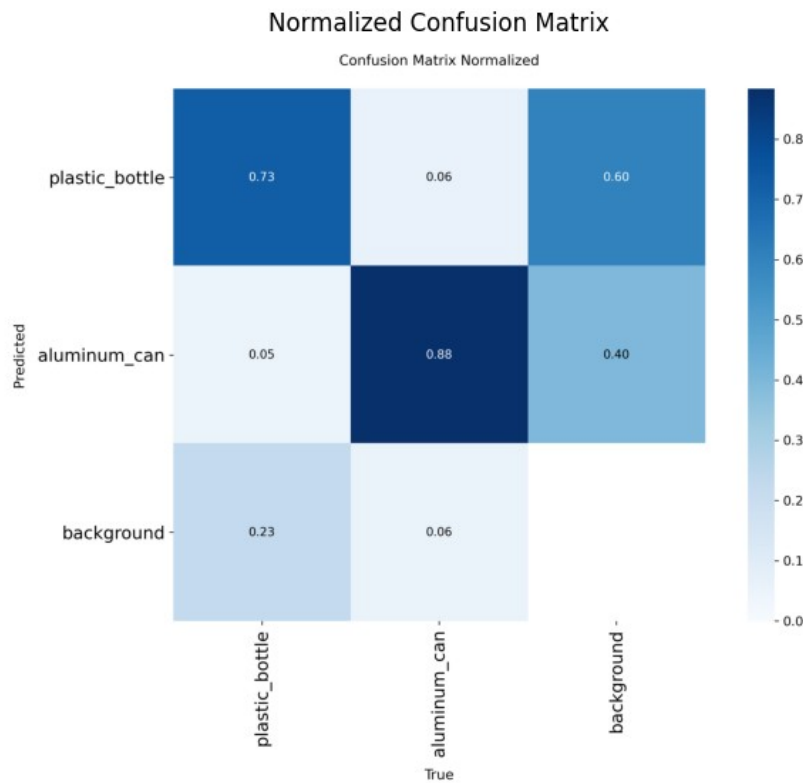


Figure 5.9: Normalized confusion matrix on the validation set.

5.14 Confidence-Threshold Analysis

The 0.65 confidence threshold used for open-set rejection was not chosen arbitrarily but from the data. Figure 5.10 shows the distribution of detection-confidence scores on the test set. Genuine detections of bottles and cans cluster at high confidence, while spurious or uncertain detections sit at low confidence, and there is a clear separation between the two populations. A threshold of 0.65 is placed in this gap: it lies above the low-confidence noise, so that non-recyclable and uncertain items fall below it and are rejected, while remaining below the mass of genuine detections, so that real bottles and cans are retained. About fifty-two per cent of raw detections clear this gate. The threshold therefore trades a small amount of recall for a large gain in the reliability of rejection, which is the correct trade-off for a machine that must not reward the wrong thing.

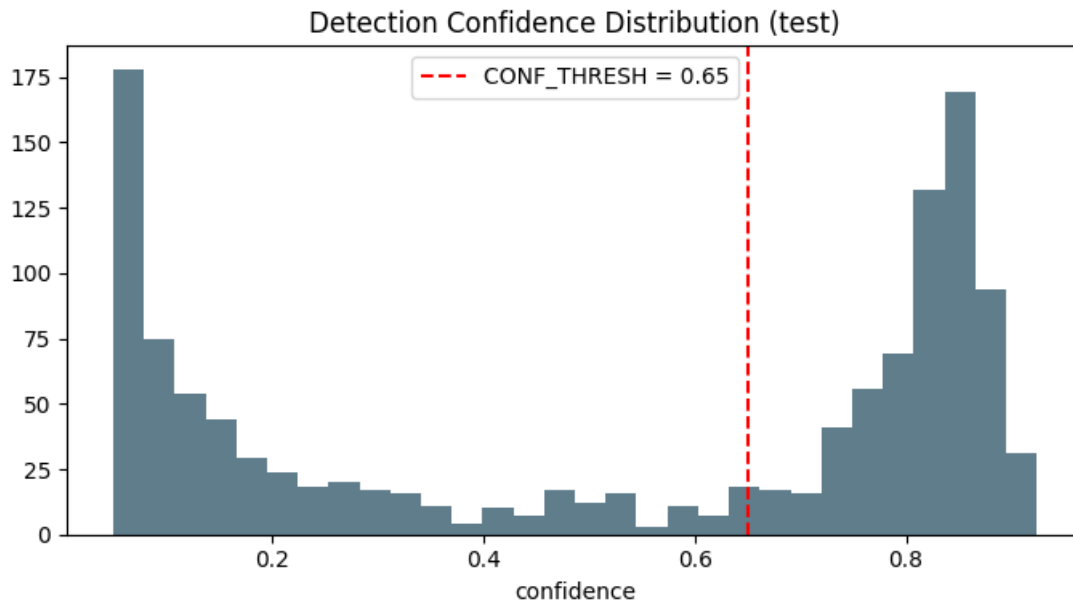


Figure 5.10: Distribution of detection confidence on the test set, with the 0.65 rejection threshold marked.

5.15 Inference Speed

Real-time operation requires that an item be detected in well under one second. On the T4 GPU the model runs at approximately 69 frames per second, a mean of about 14.5 milliseconds per image. Deployment, however, targets a laptop-class CPU, which is roughly six times slower; a conservative estimate therefore places single-image inference at about 87 milliseconds on the CPU. This is comfortably within the one-second budget and leaves ample headroom for the stability buffer described in Section 5.19, which deliberately waits for several consistent detections before accepting an item.

5.16 Qualitative Results

Beyond the aggregate metrics, the model's behaviour on individual unseen images confirms its quality. Figure 5.11 shows the model's predictions on random test images: bottles and cans are detected and correctly classified across varied backgrounds, orientations, and conditions, with tight bounding boxes and high confidence. These qualitative results match the quantitative ones and give visual confidence that the detector will perform as measured when deployed.



Figure 5.11: Model predictions on random unseen test images.

5.17 Error Analysis

No model is perfect, and understanding its failure modes is part of engineering it responsibly. The recall of 0.864 means that roughly one item in seven is not detected on the first evaluation frame, and these misses are almost entirely plastic bottles: aluminium recall is 0.97 while plastic recall is 0.76, because plastic containers vary so much more in transparency, colour, labelling, and deformation. In the field this is softened by the machine processing a continuous stream of frames rather than a single snapshot, so a briefly-missed bottle is usually caught on a later frame. The residual background confusion in the matrix corresponds to difficult presentations — heavy glare on a metallic can, a transparent bottle against a busy background, or an item only partly in view — and these are precisely the cases the confidence gate filters, since such detections tend to be low-confidence. The main systematic limitation is the domain gap between the training images and the specific view of the machine's own camera; closing this gap by collecting images from the deployed camera is the highest-value item of future work, discussed in Chapter 13.

5.18 Model Export and Optimization

The trained model was exported in two formats to serve different deployment needs. The native PyTorch format (a .pt file) is used where a full deep-learning runtime is available. For portability, the model was also exported to ONNX (Open Neural Network Exchange), a framework-neutral format of about 42.7 megabytes that runs efficiently on a CPU through a lightweight runtime without requiring PyTorch to be installed. This ONNX export is what makes it practical to run the detector on an ordinary laptop at the machine, and it opens the door to further optimization — such as quantization or acceleration on embedded neural hardware — as future work.

5.19 On-Machine Inference Logic

Accuracy on a benchmark is necessary but not sufficient; the model must behave sensibly in a live control loop. The machine reads frames continuously from its camera and runs the model on each. To turn a stream of per-frame detections into a single, trustworthy decision, a stability buffer is applied: the same class must be detected with confidence above the threshold for a sustained interval of about 1.2 seconds before the item is accepted. This eliminates two failure modes at once — momentary misfires from a single noisy frame, and double-counting of one physical item across many frames. Once an item is accepted, a cooldown and a requirement that the item leave the view prevent it from being counted again. Only after acceptance does the machine send the sorting command to the Arduino and submit the transaction to the backend. Listing 5.1 sketches this logic.

Listing 5.1: *Simplified on-machine detection-and-accept loop (Python).*

```
buffer = []                                # recent confident detections
while session_active:
    frame = camera.read()
    dets = model(frame, conf=0.65)          # YOLOv8 inference
    best = highest_confidence(dets)
    if best and best.conf >= 0.65:
        buffer.append(best.cls)
    else:
        buffer.clear()                     # reset on any gap

    if stable_for(buffer, seconds=1.2):      # same class, sustained
        cls = majority(buffer)
        arduino.write(b'P' if cls == 'plastic_bottle' else b'A')
        api.submit_detection(code, cls, MACHINE_KEY)
        buffer.clear()
        wait_until_item_leaves(camera)      # prevent double-count
```

5.20 Discussion

The artificial-intelligence module meets every goal set for it. It recognizes the two target materials with a mean Average Precision of 0.928, exceeding the 0.90 target; it rejects non-

target items reliably through confidence thresholding rather than an unwieldy “other” class; it runs in real time on commodity hardware; and it is integrated into a control loop that turns its probabilistic outputs into dependable physical actions. The design choices that made this possible — open-set rejection, the fusion of four real-box detection datasets under verified class maps, aggressive augmentation, transfer learning, a right-sized model, and a stability buffer — are, taken together, the core engineering contribution of the AI part of the project, and they are what allow a graduation-scale effort to achieve commercial-grade recognition quality on a two-class recycling problem.

5.21 Relation to Prior Work

The approach taken here is consistent with the broad trend in the literature reviewed in Chapter 2, in which convolutional detectors have displaced hand-engineered methods for waste recognition, and it extends that trend in two practically important ways. First, whereas much prior work frames waste recognition as closed-set classification over a fixed list of categories, this project treats the problem as open-set at the point of use, rejecting anything the machine is not confident is a rewarded material — a framing that matches the reality of a public machine far better than a fixed category list. Second, whereas benchmark studies often report accuracy on clean, curated images, this project deliberately fused datasets of differing quality and augmented aggressively to prepare for the messy variability of real containers, and then validated on a held-out split to estimate genuine field performance. The strong result on that unseen split, and the clean separation of the confidence distribution that enables reliable rejection, are the evidence that these choices were sound.

Chapter 6 Backend Development

The backend is the heart of the system: the single authenticated service through which every other component reads and writes data. This chapter describes the technology on which it is built, its internal structure, the way it models the domain, how it authenticates users and machines and enforces roles, how the database schema is created and seeded, the endpoints it exposes, the business logic that guarantees correct behaviour, and the way it is documented, configured, and secured.

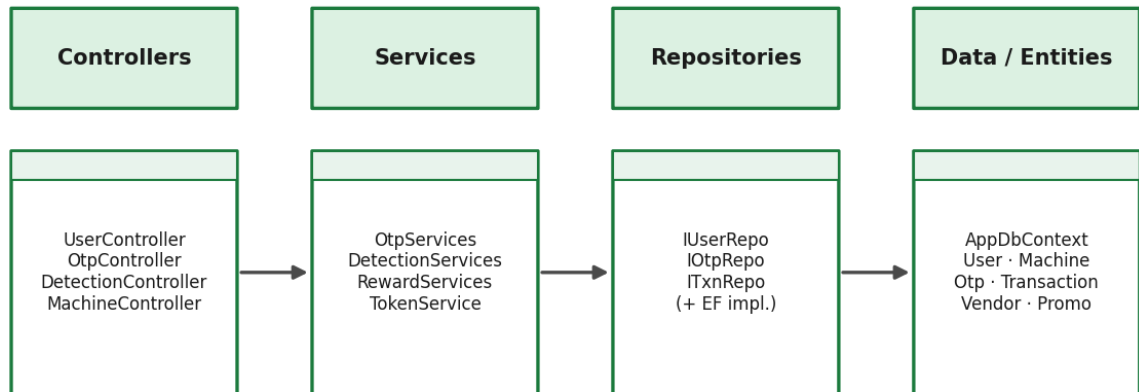
6.1 Technology Stack

The backend is an ASP.NET Core web API built on .NET 10, using Entity Framework Core as the object-relational mapper over a Microsoft SQL Server database. This stack was chosen for its maturity, performance, first-class tooling, and strong built-in support for exactly the concerns this project needs: identity management, token authentication, role-based authorization, dependency injection, and database migrations. ASP.NET Core is cross-platform and runs efficiently behind the Internet Information Services host used in deployment, and Entity Framework Core allows the schema to be expressed in C# and evolved through code-first migrations, eliminating hand-written SQL scripts.

6.2 Project Structure and Layering

As described in Chapter 4, the backend is organized into four layers — controllers, services, repositories, and the Entity Framework context with its entities. In the source tree these correspond to distinct folders: Controllers expose the HTTP endpoints; Services hold the business logic; Repositories and their interfaces encapsulate data access; a Data folder holds the DbContext, the entity classes, the migrations, and the seed logic; and a DTOs folder holds the data-transfer objects that define the request and response shapes. This structure makes the codebase navigable and enforces the dependency direction: controllers depend on services, services on repository interfaces, and only the repository implementations depend on Entity Framework. Figure 6.1 shows representative classes at each layer.

Backend Classes by Layer



Each layer depends only on the layer to its right; dependencies are injected.

Figure 6.1: Representative backend classes organized by layer.

6.3 Domain Model

The domain is expressed as a small set of entity classes that Entity Framework maps to tables. The User class extends the ASP.NET Identity user and adds the point balance. The Machine, Otp, Transaction, Vendor, Promo, and Redemption classes model the remaining concepts described in Chapter 4. Each entity carries a primary key and the appropriate foreign keys, and navigation properties express the relationships so that related data can be loaded together when needed. Keeping the domain model compact and explicit is what allows the business rules to be stated clearly in the services.

6.4 Authentication with JSON Web Tokens

Human users authenticate with JSON Web Tokens. When a user logs in with valid credentials, the backend constructs a token containing claims that identify the user and state their role, signs it with a secret symmetric key, and returns it. The client stores the token and presents it in the authorization header of every subsequent request; the backend validates the signature and the claims on each request without needing to consult the database, which keeps authentication fast and stateless. Listing 6.1 sketches the token construction.

Listing 6.1: Issuing a signed JWT on login (C#, simplified).

```
var claims = new[] {
    new Claim(ClaimTypes.NameIdentifier, user.Id),
    new Claim(ClaimTypes.Name, user.UserName),
```

```

    new Claim(ClaimTypes.Role, role)           // "User" or "Admin"
  };
  var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSecret));
  var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
  var token = new JwtSecurityToken(
    issuer: cfg["Jwt:Issuer"], audience: cfg["Jwt:Audience"],
    claims: claims, expires: DateTime.UtcNow.AddDays(7), signingCredentials: creds);
  return new JwtSecurityTokenHandler().WriteToken(token);

```

Figure 6.2 shows the full authentication flow, from the login exchange that issues the token to the validation of that token on every subsequent request.

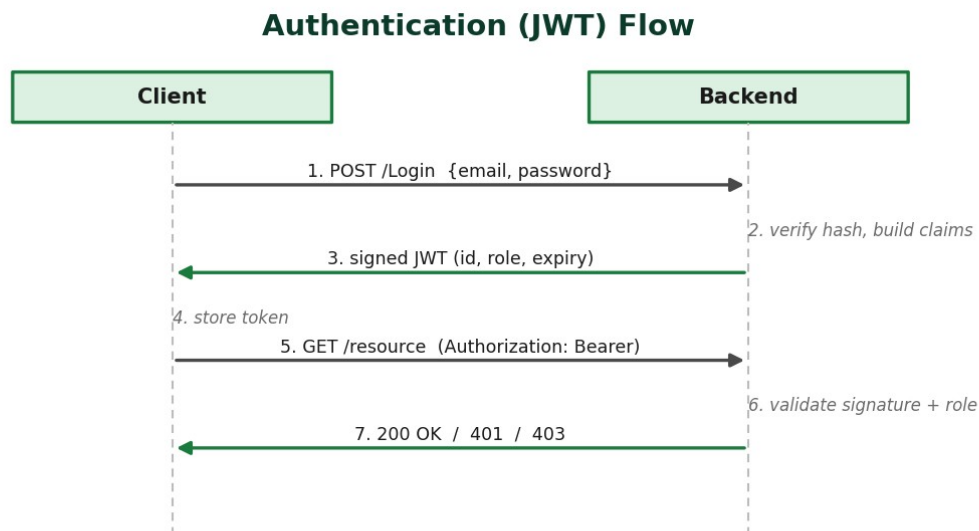


Figure 6.2: *The JSON Web Token authentication flow.*

6.5 Authorization and Roles

Authorization is role-based. Two roles exist, User and Admin, seeded at startup. Endpoints are decorated with authorization attributes that require a specific role, so the framework refuses a request from a user who lacks the required role before any controller code runs, returning HTTP 403. This cleanly separates the two capability sets: an ordinary user can generate pairing codes, view their history, and redeem rewards, but cannot touch machines or promotions; an administrator can manage machines, vendors, and promotions, but is deliberately routed away from the user reward flow. The separation is enforced centrally by the framework rather than scattered through the code, which makes it both reliable and auditable.

6.6 The Machine Key

The machine is not a human user and does not log in with a password; it authenticates with a shared secret key transmitted in a custom HTTP header on every detection submission. The backend checks this key before accepting any transaction, so that only genuine machines can

credit points. This second, separate credential is important: it means that even a fully authenticated ordinary user cannot forge a detection and award themselves points, because they do not possess the machine key. The key is supplied to the backend and the machine through configuration and can be rotated without code changes.

6.7 Database Migrations and Seeding

The database schema is defined in code and evolved through Entity Framework Core migrations. When the backend starts against a fresh database, it applies all pending migrations automatically, creating the tables, keys, and indexes without any manual SQL. At startup the system also seeds the two roles and a default administrator account whose credentials are read from configuration, so that the deployed system is immediately usable and the administrator password can differ between environments and be rotated safely. The seeding logic is written to be idempotent — safe to run on every start — and to rotate the administrator password to the configured value if it has changed, using the Identity password-management API rather than any unsupported operation.

6.8 Controllers and Endpoints

The API is organized into controllers by resource. Table 6.1 lists the principal endpoints; the complete reference, with request and response shapes, appears in Appendix A. Together they implement registration and login, the pairing and session lifecycle, detection submission, transaction history and reporting, machine management, and the rewards catalogue and its redemptions.

Table 6.1: *Principal API endpoints by controller.*

Controller	Representative endpoints	Auth
User	Register, Login, Coins	Public / JWT
Otp	Generate, Pair, EndSession	JWT (User)
Detection	session/start, status, total, submit	Machine key
Transaction	User, Admin/{machineId}	JWT (User / Admin)
Machine	GET, POST, PUT, DELETE	JWT (Admin)
Vendor / Promo / Redemption	list, add, redeem	JWT (Admin / User)

6.9 Business Logic and Correctness

The services enforce the rules that make the system trustworthy. The generation of a pairing code illustrates the care taken: a code is drawn at random and checked against the currently active codes, and the draw repeats until a value is produced that does not collide, guaranteeing

that every active code is unique at any instant (Listing 6.2). Resolving a submitted detection to a user is equally strict: the code must exist, be active, be unexpired, and be bound to a user, or the submission is refused; when it succeeds, the item is credited to exactly that one user. Ending a session sets the code's status to stopped, which the machine observes on its next status poll. Redemption checks that the user's balance covers the reward's cost before deducting points and revealing the code. Each of these rules is exercised by the automated tests of Chapter 10.

Listing 6.2: *Generating a globally-unique pairing code (C#, simplified).*

```
string code;
do {
    code = Random.Shared.Next(0, 1_000_000).ToString("D6");
} while (await _db.Otps.AnyAsync(o =>
    o.Code == code && o.Status != OtpStatus.Stopped));
// code is now unique among all active sessions
```

6.10 Error Handling and Validation

The backend validates every request and responds with the appropriate HTTP status code and a clear message rather than failing opaquely. Malformed requests receive 400; unauthenticated requests receive 401; authenticated requests lacking the required role receive 403; and requests for missing resources receive 404. Invalid domain operations — an unknown item type, a detection without the machine key, a redemption beyond the user's balance — are rejected explicitly. This disciplined error handling is what allows the clients to present helpful messages and the automated tests to assert on precise outcomes.

6.11 API Documentation

The API is described by an OpenAPI (Swagger) document generated from the code and served as an interactive page. This documentation lists every endpoint, its parameters, and its request and response schemas, and it allows a developer to authenticate and invoke endpoints directly from the browser. Serving it live — in every environment, not only during development — made integration between the team's components and the writing of the automated tests substantially easier, because the contract was always visible and executable.

6.12 Configuration and Secrets

All environment-specific values and secrets — the database connection string, the JWT signing key, the machine key, and the administrator credentials — are supplied through configuration rather than being hard-coded, following the twelve-factor principle of strict separation of configuration from code. This makes the same build deployable to different environments and

allows every secret to be rotated without recompiling. The handling of these secrets in the deployed environment is described in Chapter 11.

6.13 Threat Model

The backend was designed against a simple threat model. Impersonation of a user is prevented by signed tokens that cannot be forged without the secret key. Forging a detection to award points is prevented by the separate machine key. Guessing an active pairing code is impractical because codes are short-lived and only a small number are active at once, and a code that is already paired cannot be re-paired. Interception of traffic is prevented by HTTPS in deployment. Exposure of secrets is mitigated by keeping them in configuration and rotating them before public release. These measures are proportionate to a prototype that nonetheless is deployed on the public internet, and the residual risks are noted in Chapter 12.

6.14 Data Transfer Objects and Validation

The shapes exchanged over the API are defined by data-transfer objects (DTOs) that are deliberately separate from the internal entity classes. This separation has two benefits. First, it decouples the public contract from the database schema, so the schema can evolve without breaking clients and, conversely, internal fields are never inadvertently exposed — a login response, for instance, carries a token and the point balance but never the password hash. Second, DTOs are the natural place to declare validation rules, so that malformed input is rejected at the boundary before it reaches the business logic. Listing 6.3 shows a representative pair of request and response DTOs.

Listing 6.3: *Representative request and response DTOs (C#).*

```
public record LoginRequest(string Email, string Password);

public record LoginResponse(string Token, string Role, int Coins);

public record SubmitDetectionRequest(string Code, string ItemType);
```

6.15 Logging and Diagnostics

The backend uses the built-in structured logging of ASP.NET Core to record significant events — authentication failures, rejected submissions, and unhandled exceptions — at appropriate severity levels. In development these logs stream to the console; in deployment they are available through the host. Combined with the live API documentation and the automated test suites, this logging provides enough visibility to diagnose problems quickly without the weight of a full observability stack, which is proportionate to the scale of the system.

6.16 Summary

The backend is the component on which every other part of the system depends, and it was built to be secure, correct, and maintainable. Its layered structure separates the concerns of handling HTTP, executing business logic, and accessing data; its two authentication mechanisms protect the two kinds of client; its migrations and seeding make it deployable to a fresh environment with no manual setup; and its business rules — unique code generation, strict session resolution, and balance-checked redemption — are what make the reward economy trustworthy. Because it concentrates all of this behind a single documented API, the machine, the mobile app, and the web app can all remain thin, and the guarantees the backend enforces hold uniformly no matter which client makes the request.

Chapter 7 The Mobile Application

The mobile application is the user's reward companion — the surface through which a person registers, pairs with a machine, watches points accrue, and redeems rewards. This chapter describes the framework and structure of the application, its screens, the service layer through which it talks to the backend, the QR-scanning and camera subsystem (including a subtle defect that was diagnosed and fixed), the real-time session view, the routing that gives administrators a separate interface, the in-app server configuration, the handling of local time, and the way the application is branded and distributed.

7.1 Framework and Rationale

The application is built with Flutter, Google's cross-platform user-interface framework, using the Dart language and Material 3 design. Flutter was chosen because it produces a single, high-performance, natively-compiled application from one codebase, with a rich widget library and excellent support for exactly the features this project needs: camera access, secure storage of the authentication token, HTTP networking, and internationalized date and time formatting. Its hot-reload development cycle also accelerated the iterative methodology described in Chapter 1. Figure 7.1 shows the sign-in screen, which shares its brand with the web application.

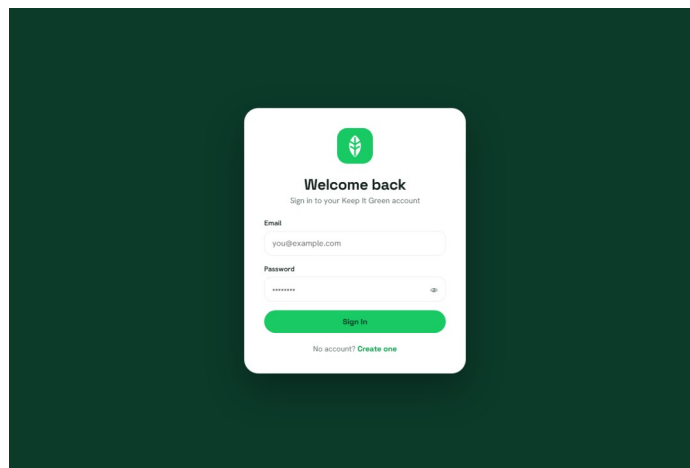


Figure 7.1: The Keep It Green sign-in screen (brand shared across app and web).

7.2 Application Structure

The application comprises fourteen screens, including a welcome screen, login and registration, a home dashboard with tabbed navigation, a QR-scanning screen, a live-session screen, a rewards catalogue, a redemption screen, a transaction-history view, a profile screen, a dedicated administrator home, and an in-app server-settings screen. State is kept deliberately simple and local to each screen, and all communication with the backend is funnelled through

a single service module, so that concerns are separated: the screens present and collect information, while the service handles the network. This structure keeps the application easy to reason about and to extend.

7.2.1 Screen-by-Screen Walkthrough

Table 7.1 walks through the application's screens and the purpose of each, giving a complete picture of the user's journey through the app from first launch to reward redemption.

Table 7.1: *The application's screens and their purpose.*

Screen	Purpose
Welcome	Brand splash and entry to sign-in or registration
Register	Create an account (email, password)
Login	Authenticate; obtain and store the JWT
Home (tabs)	Dashboard with balance and navigation to features
Scan	Scan the machine's QR code or enter the six-digit code
Live Session	Real-time points as items are accepted; Finish button
Rewards	Catalogue of vendor promotions (codes hidden)
Redeem	Spend points; reveal the promo code on success
History	List of transactions and redemptions (Cairo time)
Profile	Account details and sign-out
Admin Home	Administrator dashboard (shown for Admin role)
Manage Machines	Add, enable, disable, delete machines (admin)
Manage Rewards	Create vendors and promotions (admin)
Server Settings	Set/override the backend address in-app

7.3 The API Service Layer

A single Dart service class encapsulates every call to the backend. It holds the base URL, attaches the stored JSON Web Token to each authenticated request, serializes and deserializes JSON, and translates network and server errors into friendly messages for the user interface. Centralizing this logic means that cross-cutting concerns — authentication, the server address, error handling, and time conversion — are implemented once and applied everywhere. Listing 7.1 sketches an authenticated call.

Listing 7.1: *An authenticated backend call from the app (Dart, simplified).*

```
Future<int> fetchCoins() async {
  final res = await http.get(
    Uri.parse('$baseUrl/api/User/Coins'),
    headers: { 'Authorization': 'Bearer $_token' },
  );
  if (res.statusCode == 200) return jsonDecode(res.body)['coins'];
  throw ApiException(_friendlyMessage(res));
}
```

```
}
```

7.4 Core User Features

- **Pairing:** the user scans the machine's QR code or types the six-digit code to open a session and bind it to their account.
- **Live session:** points appear in real time as items are accepted, with the running total and the most recent item displayed, so the user trusts that the machine counted correctly.
- **Rewards:** a catalogue of vendor promotions is presented, and the promotional code is hidden until the user redeems it, preventing codes from leaking to people who have not spent points.
- **History:** every recycling transaction and every redemption is listed, in Cairo local time.
- **Finish:** finishing the session calls the backend so the machine returns to its idle state for the next person.

7.5 QR Scanning and the Camera Subsystem

Pairing by QR code requires camera access, which proved to be the most demanding engineering detail of the application. The scanner is built on a camera-scanning library, and the application explicitly requests the camera permission at runtime before starting the preview, presenting a clear diagnostic if permission is denied or the camera cannot start. During development a defect surfaced in which, on some devices, the camera failed to open and reported a low-level null-reference error originating inside the underlying machine-learning barcode component. The root cause was identified precisely: the scanning library bundles its barcode model by default, but an additional configuration entry that had been added for an alternative, unbundled delivery path caused the component to look for a module that did not exist, producing the null reference. Removing that configuration entry resolved the failure, and the corrected build opens the camera reliably. A manual code-entry fallback is provided regardless, so pairing never depends solely on the camera.

7.6 The Administrator Application

Because the backend restricts the user reward endpoints to the user role, an administrator who signs in is routed to a dedicated administrative interface rather than the ordinary reward screens. The routing decision is made from the role claim carried inside the authentication token, which the application decodes after login. From the administrator interface, a privileged user can add,

enable, disable, and delete machines; create vendors and promotional codes; and view per-machine transaction reports — the same capabilities available on the web dashboard, delivered natively on the phone.

7.7 Server Configuration and Resilience

Because a demonstration backend may move between networks and hosts, the application includes an in-app server-address setting, so the backend it targets can be changed without rebuilding, and it automatically discards a stale saved address when the application is upgraded, so it always reaches the current backend. By default it targets the public deployed backend over HTTPS. This small piece of resilience proved valuable during development and demonstrations, when the backend's location changed several times.

7.8 Localization and Time

Consistent with the rest of the system, the application converts every timestamp it receives from Coordinated Universal Time to Cairo local time before displaying it, using a time-zone library with correct daylight-saving handling. Dates and times are then formatted according to locale conventions. As a result, a user always sees the moment an item was recycled or a reward redeemed in their own local time, regardless of the server's location.

7.9 Branding and Distribution

The application carries the Keep It Green identity throughout, and its launcher icon is the project's green leaf, generated at every required Android density. It is distributed as a versioned, installable Android package (an APK) that any Android user can side-load to try the system, configured to reach the deployed backend over a secure connection. Building the release package is a single command in the Flutter toolchain, and the resulting file is what was tested end-to-end against the live backend as reported in Chapter 10.

7.10 Testing the Application

The application was validated at several levels. The Flutter static analyzer runs over the entire codebase and reports no issues, catching type errors and unsafe patterns before the app is ever run. The application's calls to the backend were exercised against the live deployed API, so the network layer, the authentication handling, and the parsing of every response were tested against the real server rather than a mock. And the assembled application was used by hand across its full journey — register, log in, pair, recycle, watch points accrue, finish, and redeem — on a real device against a real machine, which is the ultimate test of whether the parts cohere.

This last, manual testing was where the camera defect described in Section 7.5 was found and, after correction, confirmed resolved, and where the real-time updating of the live-session screen and the hiding of promotional codes until redemption were verified from the user's point of view. Because the application is a client of the same backend that the automated suites of Chapter 10 exercise, much of its behaviour is also validated indirectly by those suites, which assert on the very endpoints the application calls.

Chapter 8 The Web Application and Machine Kiosk

Two further user surfaces complete the system: a web application that offers the same capabilities as the mobile app in any browser, and the on-machine kiosk interface that a user interacts with while physically at the machine. This chapter describes both, including the deliberately lightweight design of the web application, its behaviour as an installable progressive web app, the way it is served directly by the backend, and the states and implementation of the kiosk.

8.1 The Web Application

The web application provides registration, login, the live session, the rewards catalogue and redemption, history, and the full administrator dashboard, all in the browser. It is deliberately lightweight: a single-page application written in plain JavaScript with a hash-based router and no build step, which keeps the delivered payload tiny, the behaviour transparent, and the maintenance burden low. It shares the Keep It Green brand — the green palette and the display typeface — with the mobile application, and it supports a dark theme and Cairo local time. Table 8.1 lists its client-side routes.

Table 8.1: *Web application routes (hash-based).*

Route	View
#/login	Sign in
#/register	Create account
#/home	Dashboard and balance
#/session	Live recycling session
#/rewards	Rewards catalogue and redemption
#/history	Transactions and redemptions
#/admin	Administrator dashboard (Admin role)

8.2 Progressive Web App

The web application is also an installable progressive web app. A web manifest describes its name, icons, and colours so that it can be added to a phone's home screen and launched like a native application, and a service worker caches the application shell so that it loads quickly and remains available through brief network interruptions. The service worker is versioned, so that when the application is updated the new version is picked up cleanly rather than being masked by a stale cache — a lesson applied after an early caching issue during development.

8.3 Serving from the Backend

The web application and the kiosk interface are served as static files directly by the backend. This means the entire user-facing system deploys as a single unit: there is no separate web server or content-delivery step to configure, the front-end is always served from the same origin as the API it calls (which avoids cross-origin complications), and a single deployment updates both the service and its interfaces together. Static responses are served with cache headers chosen so that updated files are not masked by the browser cache.

8.4 The Machine Kiosk Interface

The kiosk is the screen the user sees at the machine, and it is designed to be understood at a glance, in the manner of an automated teller machine. It moves through a small number of clear states. In the idle state it shows the pairing QR code and the six-digit code alongside a keypad, inviting the user to pair. Once a user pairs, it shows a live camera view with an on-item detection frame and the instruction to place an item, shown in Figure 8.1. After each accepted item it shows a confirmation, and then asks whether the user wishes to add another item or to finish. On finishing, it presents a summary of the points earned in the session and the user's new balance, shown in Figure 8.2, before returning to the idle state for the next person.

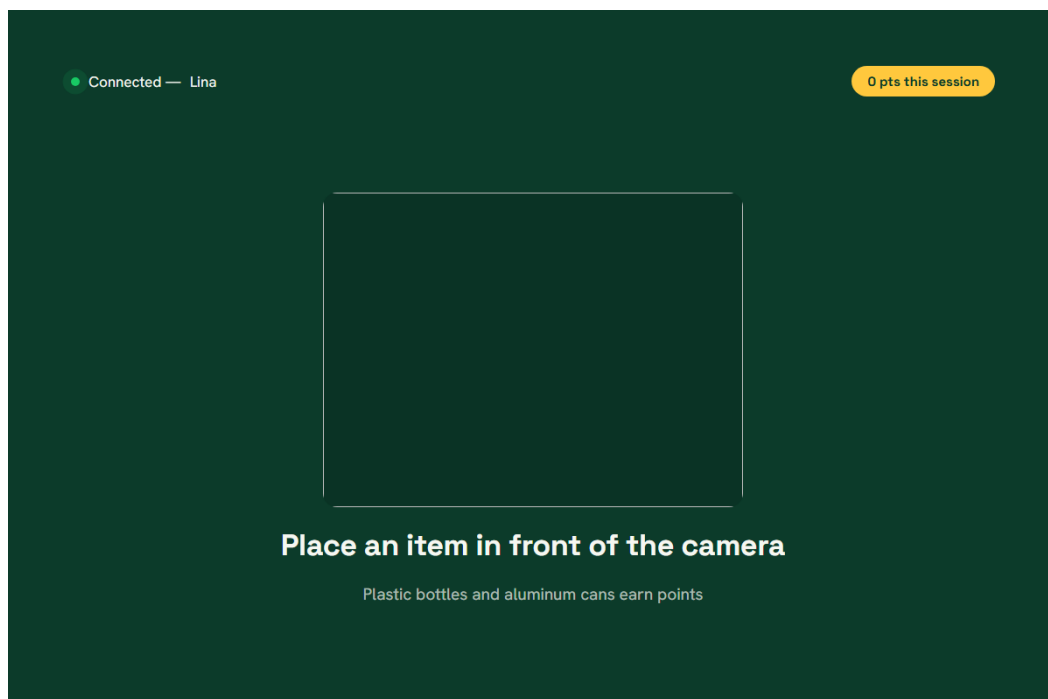


Figure 8.1: *Kiosk — the place-an-item state with the live camera view and detection frame.*

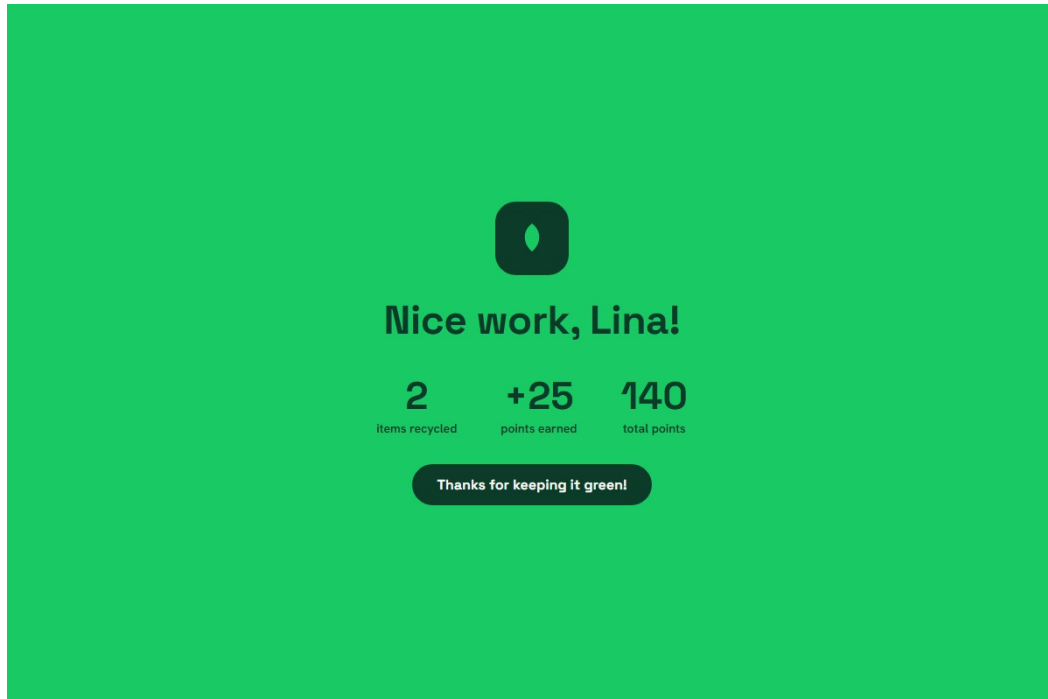


Figure 8.2: Kiosk — the reward-summary state shown when a session finishes.

The kiosk's behaviour is a small state machine, shown in Figure 8.3: it moves from idle to place-item on pairing, to a confirmation on each accepted item, then either back to place-item to add another or on to the summary when the user finishes, and finally back to idle.

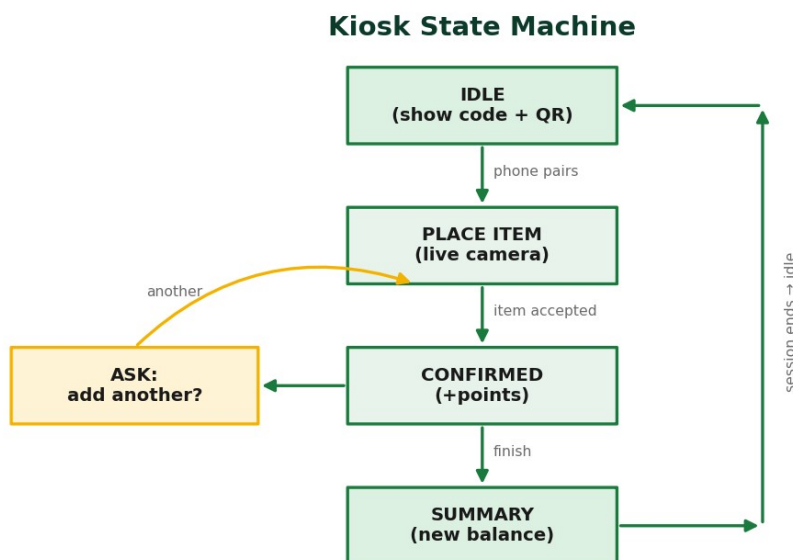


Figure 8.3: State machine of the kiosk interface.

8.5 Kiosk Implementation and Integration

The entire kiosk runs as a single program on the machine's laptop. A lightweight desktop web-view host (pywebview) displays the web-based kiosk interface in a full-screen window, while

the same Python process runs the YOLO detection model on the webcam and communicates with the Arduino over the serial link. Housing the interface, the detection, and the hardware control in one process keeps the machine software self-contained and simple to launch, and it allows the interface to react immediately to detection events and session changes. The kiosk polls the backend for the session status so that when the user finishes on their phone, or the session otherwise ends, the machine returns to idle without manual intervention.

8.6 Consistency Across Surfaces

The mobile application, the web application, and the kiosk deliberately share one brand, one backend, and one set of concepts — sessions, points, rewards, and history — so that a user moving between the machine screen and their phone experiences a single coherent system rather than three disconnected tools. This consistency is a direct benefit of the architectural principle that all three are thin clients of the same authenticated API.

8.7 Accessibility, Responsiveness, and Branding

The web surfaces were built with attention to accessibility and responsiveness. The layout adapts from a phone-sized screen to a desktop browser, so the same application is usable on the device a person happens to have; touch targets are large enough to use comfortably; and the colour palette maintains legible contrast, with a dark theme available for low-light conditions and for user preference. The kiosk, in particular, uses large type and high-contrast, one-decision-at-a-time screens precisely so that it can be used quickly and without instruction by anyone who approaches the machine.

Across all of these surfaces the Keep It Green identity is applied consistently — the forest-and-spring green palette, the sunlit accent, and the distinctive display typeface — so that the machine, the app, and the web feel like one product. This consistency is not merely cosmetic: a coherent, recognizable identity builds the trust that a reward system depends on, reassuring the user that the machine in front of them and the app in their hand are part of the same trustworthy system.

Chapter 9 Hardware and Machine Integration

The physical machine is where the system meets the real world: it accepts an item, recognizes it, and sorts it. This chapter describes the machine's hardware — its computing, sensing, and actuation — the simple and robust protocol by which the software commands the hardware, the pipeline that turns a detection into a physical sort, the industrial and prototype design of the enclosure and bins, and the considerations of assembly, safety, and reliability.

9.1 Overview and Bill of Materials

The machine is built from inexpensive, commodity components, which is central to its value proposition. A laptop-class computer runs the detection software and hosts the kiosk; an ordinary webcam provides the images; an Arduino microcontroller drives two servo motors that open the plastic and aluminium compartments; and an enclosure with internal sorting bins houses everything. Table 9.1 lists the principal components. The entire actuation controller costs only a few dollars, and the most expensive element, the computer, can in a production version be replaced by a low-cost single-board computer.

Table 9.1: *Principal hardware components (prototype).*

Component	Role	Notes
Laptop / mini-PC	Runs detection + kiosk	Commodity; replaceable by single-board PC
USB webcam	Captures item images	Standard resolution sufficient
Arduino board	Drives sorting servos	Low-cost microcontroller
2 × servo motors	Open plastic / aluminium gates	One per compartment
Enclosure + bins	Houses and sorts	3D-designed, brand-coloured

9.2 The Microcontroller and Actuation

The Arduino microcontroller is responsible for the real-time physical action. It is a natural division of labour: the computer performs the heavy visual computation, which a microcontroller could not, while the microcontroller performs precise, reliable, real-time control of the servo motors, for which it is ideally suited and which frees the computer from timing-critical work. Each of the two servos controls a gate or flap that, when opened, directs the falling item into the correct compartment and then closes again, ready for the next item.

9.3 The Serial Protocol

Communication between the detection software and the Arduino uses a minimal, robust, one-byte serial protocol. When the software confirms a plastic bottle it sends the single byte ‘P’,

and the Arduino opens the plastic gate; when it confirms an aluminium can it sends ‘A’, and the Arduino opens the aluminium gate. There is no need for a more elaborate protocol, and the simplicity is a virtue: there is very little that can go wrong, the link is easy to debug, and the latency is negligible. Table 9.2 states the protocol, and Listing 9.1 shows the core of the Arduino program that implements it.

Table 9.2: *The one-byte serial sorting protocol.*

Detection	Byte	Arduino action
plastic_bottle	P	Open the plastic gate, pause, close
aluminum_can	A	Open the aluminium gate, pause, close

Listing 9.1: *Core of the Arduino sorting sketch (C++).*

```
#include <Servo.h>
Servo plastic, aluminium;
void setup() {
  Serial.begin(9600);
  plastic.attach(9); aluminium.attach(10);
  plastic.write(0); aluminium.write(0); // gates closed
}
void loop() {
  if (Serial.available()) {
    char c = Serial.read();
    Servo &gate = (c == 'P') ? plastic : aluminium;
    if (c == 'P' || c == 'A') {
      gate.write(90); delay(800); gate.write(0); // open, then close
    }
  }
}
```

9.4 From Detection to Physical Sort

The full pipeline that turns an inserted item into a sorted one ties together the AI module of Chapter 5 and the hardware of this chapter. The webcam streams frames to the computer; the model detects and classifies the item; the stability buffer waits until the detection is confident and sustained, which both prevents misfires and ensures a single physical item is counted only once; the software then sends the appropriate byte to the Arduino, which opens the correct gate so the item falls into the right compartment; and in parallel the software submits the transaction to the backend so the user is credited. The whole sequence, from a settled item to a completed sort and credit, takes little more than a second.

9.5 Industrial and Prototype Design

The machine's enclosure and its internal sorting bins were modelled in three dimensions, coloured in the project's brand palette, and designed with parts suited to fabrication by three-dimensional printing or CNC machining, so that the design is genuinely manufacturable rather than merely a render. Figure 9.1 shows the front of the machine, and Figure 9.2 the interior of the two sorting compartments. The design places the camera and intake at a comfortable height,

routes an accepted item to the correct bin, and presents the kiosk screen prominently, embodying the ATM-like interaction the software provides.



Figure 9.1: Front view of the machine enclosure (3D design).



Figure 9.2: Interior of the plastic and aluminium sorting compartments.

9.6 Assembly, Safety, and Reliability

In the prototype the components are assembled into the enclosure with the camera positioned to view the intake, the servos mounted to drive the sorting gates, and the Arduino connected to

the computer by USB, which also powers it. Reliability is addressed primarily in software, through the stability buffer and the requirement that an item leave the view before another is counted, which together prevent the most likely failure — miscounting. Safety in a supervised prototype is straightforward, with no exposed moving parts capable of harm and low-voltage electronics throughout. Hardening the machine for unsupervised, outdoor operation — sealing it against weather, securing it against tampering, and adding a weight or fill sensor — is identified as future work in Chapter 13.

9.7 Wiring and Power

The prototype's electrical arrangement is deliberately simple. The webcam and the Arduino connect to the computer by USB, and the USB connection also supplies power to the microcontroller, so no separate power supply is required for the control electronics at prototype scale. Each servo's three leads — power, ground, and signal — connect to the Arduino, with the signal leads on the pins the sketch of Listing 9.1 drives. For a production unit the servos would be powered from a dedicated regulated supply rather than from the USB bus, because several servos actuating together can draw more current than a USB port should provide; this is a standard refinement and does not change the control logic. Grounds are held common so that the serial signalling is reliable.

9.8 Calibration and Tuning

Two small calibrations make the hardware behave well. The open and closed angles of each servo are set so that a gate opens far enough to divert an item cleanly and closes fully to separate the compartments, and the open interval is tuned so that an item has time to fall through before the gate closes. On the software side, the stability interval and confidence threshold described in Chapter 5 are the tuning points that govern when a detection becomes an action; together with the servo timing they determine the overall responsiveness and reliability of the sort. These parameters were set empirically during integration and are exposed in configuration so they can be adjusted without changing code.

9.9 Summary

The hardware demonstrates that the intelligence of the machine can be turned into a physical action reliably and at very low cost. A commodity computer and webcam perform the recognition; a few dollars of microcontroller and servos perform the sorting; and a one-byte serial protocol connects the two with almost nothing that can go wrong. The stability buffer in software, together with the simple, robust actuation in hardware, is what allows the prototype

to sort dependably. The same design, hardened and powered from a dedicated supply, scales directly to a production unit, so the prototype is not a throwaway demonstration but a faithful, smaller-scale version of what a deployed machine would be.

Chapter 10 Testing and Validation

A system is only as trustworthy as the evidence that it works. This chapter describes how Keep It Green was tested and validated, from the strategy and the levels of testing, through the automated end-to-end suites that exercise the real backend exactly as the machine, phone, and browser do, to the testing of the artificial-intelligence model, the negative and security tests, the performance and static-analysis checks, and finally the mapping of tests back to requirements that demonstrates coverage.

10.1 Testing Strategy

Testing was organized around several levels. Unit-level checks validated individual pieces of business logic. Integration testing verified that the layers of the backend worked together against a real database. End-to-end (system) testing, which forms the backbone of the validation, drove complete user journeys against the running backend through its public API, exactly as a real client would, so that what was tested was the integrated system rather than isolated parts. The model was validated separately against a held-out test set as described in Chapter 5. Finally, informal user-acceptance testing confirmed that the assembled system behaved sensibly in the hands of real users. Both positive paths and negative paths were tested throughout, because a system that must not reward the wrong thing has to be judged as much on what it refuses as on what it accepts.

10.2 Test Environment

The end-to-end suites were written as scripts that call the backend over HTTP, and they were run against two environments: a local instance during development, and, decisively, the live deployed backend, so that the reported results reflect the system as the public actually encounters it. Running against the live environment also validated the deployment itself — the database, the HTTPS termination, and the configuration — not merely the application code.

10.3 Core Recycling Flow

The core suite performs a complete recycling journey and asserts on every step, twenty-three assertions in all. It registers a fresh user, logs in and obtains a token, and confirms the starting balance is zero. It lists machines, starts a machine session, and confirms the session is initially unpaired. It pairs the user and confirms the session is now paired. It submits a plastic detection and an aluminium detection, and confirms that the session total and the user's balance are both twenty-five points — ten for the bottle plus fifteen for the can. It reads the transaction history and confirms both items appear. It then redeems an affordable reward and confirms that the

points were deducted and a redemption recorded. Table 10.1 summarizes the checks. All twenty-three pass.

Table 10.1: *Core recycling-flow checks (23 assertions).*

Step	Assertion	Result
Register / login	Account created; token issued; balance = 0	Pass
List machines	Machine list returned	Pass
Start session	Unique code issued; status unpaired	Pass
Pair	Code bound to user; status paired	Pass
Submit plastic (+10)	Transaction recorded; total = 10	Pass
Submit aluminium (+15)	Transaction recorded; total = 25	Pass
Balance / history	Balance = 25; both items in history	Pass
Redeem reward	Points deducted; redemption recorded	Pass

10.4 Administrative Flow

A second suite validates the administrative capabilities, twelve assertions in all. It logs in as the administrator and confirms that the token carries the administrator role. It then adds a machine, toggles its availability, reads its transaction report, and deletes it; and it adds a vendor and a promotional code. Finally, and importantly, it confirms that an ordinary user is correctly forbidden from every administrative endpoint, receiving HTTP 403. All twelve checks pass, and the last of them is as valuable as the rest, because it proves that the role separation actually holds.

Table 10.2: *Administrative-flow checks (12 assertions).*

Area	Assertion	Result
Admin login	Token carries Admin role	Pass
Machine CRUD	Add, toggle, report, delete succeed	Pass
Vendor / promo	Vendor and promo created	Pass
Role enforcement	Ordinary user refused (403) on all admin endpoints	Pass

10.5 Multi-User Session Isolation

Because a machine is shared, the most safety-critical property of the whole system is that concurrent users never interfere. A dedicated suite of thirteen checks creates two users on the same machine and verifies the guarantees stated in Chapter 4: each session receives a unique code; each user is credited only for their own items; each code's session total resolves to the correct user; the two codes map to different users; and a code already paired to one user cannot be re-paired or hijacked by another. All thirteen checks pass, which is the empirical confirmation that the ATM-style session model is correct.

Table 10.3: *Multi-user isolation checks (13 assertions).*

Property	Assertion	Result
Code uniqueness	Two concurrent sessions get different codes	Pass
Correct attribution	Each user credited only for their own items	Pass
Session resolution	Each code's total resolves to the right user	Pass
One-to-one mapping	The two codes map to different users	Pass
No hijacking	A paired code cannot be re-paired by another user	Pass

10.6 Model Testing

The artificial-intelligence model was validated on its held-out test set as detailed in Chapter 5, achieving a mean Average Precision of 0.928 against a target of 0.90, with precision 0.924 and recall 0.864 — aluminium at 0.986 and plastic at 0.870 per class. Because the test set was isolated from training and epoch selection, these figures constitute an unbiased estimate of field performance rather than a measurement on familiar data.

10.7 Negative and Security Testing

The negative tests deliberately try to make the system misbehave and confirm that it refuses. A detection submitted with an unknown item type is rejected. A detection submitted without the machine key is rejected, proving that points cannot be forged by an ordinary user. A redemption for more points than the user holds is rejected. An unauthenticated request to a protected endpoint is refused with 401, and an authenticated request lacking the required role with 403. These tests are woven through the suites above and are as important as the positive paths, because they validate exactly the guarantees on which the integrity of the reward economy depends.

10.8 Performance and Static Analysis

Performance was checked against the real-time requirement: model inference is well under the one-second budget, as reported in Chapter 5, and the backend responds to the API calls in the interactive range under the modest load of the test suites. Static analysis complemented the dynamic tests: the mobile application passes the Flutter analyzer with no reported issues, and the backend compiles cleanly. Static analysis catches a different class of defect from runtime testing — unused code, unsafe patterns, type errors — and a clean result on both raises overall confidence.

10.9 User-Acceptance Testing

Finally, the assembled system was exercised informally by real users pairing a phone with the machine, recycling items, watching points accrue, and redeeming rewards. This acceptance testing surfaced usability observations rather than functional defects — for example, it confirmed that the ATM-style kiosk flow was understood without instruction and that the promotional code was correctly hidden until redemption — and it validated that the parts, individually tested, cohere into a system that people can actually use.

10.9.1 Defect Log

Testing, both automated and manual, surfaced defects that were diagnosed and fixed during development; recording them is part of an honest account of the engineering. Table 10.5 lists the most significant. Each was resolved and re-tested, and each is covered by the suites above or by the corrected build.

Table 10.5 — Selected defects found and fixed

Defect	Where	Resolution
Camera failed to open (null reference)	Mobile app	Removed a conflicting ML-Kit configuration entry
Promo code shown before redemption	Web rewards	Hid the code until a redemption is recorded
Startup crash after deploy (500.30)	Backend seeding	Rotated admin password via supported API + guard
Upload archive would not extract	Deployment	Repackaged with forward-slash entries; stripped cruft
Stale web assets after update	Web PWA	Versioned the service worker and cache

10.10 Requirements Traceability

Taken together, the suites trace back to and satisfy the requirements of Chapter 3. The core suite exercises the end-user functional requirements; the administrative suite exercises the administrator requirements and the role-based security requirement; the isolation suite validates the reliability requirement that each item is credited to exactly one user; the model evaluation validates the accuracy requirement; and the performance checks validate the real-time requirement. Table 10.4 summarizes the results across all suites.

Table 10.4: *Summary of automated end-to-end results.*

Suite	Checks	Result
Core recycling flow	23	23 / 23 pass
Administrative flow	12	12 / 12 pass
Multi-user isolation	13	13 / 13 pass

Suite	Checks	Result
Total	48	48 / 48 pass

10.11 Summary of Testing

Taken as a whole, the testing gives strong, concrete evidence that the system works as specified. Forty-eight automated end-to-end checks pass against the live deployed backend, exercising the complete user journey, the administrative capabilities, and — most importantly for a shared machine — the isolation between concurrent users; the detection model exceeds its accuracy target on unseen data; the negative and security tests confirm that the system refuses exactly what it must refuse; the performance checks confirm the real-time budget is met; and static analysis reports no issues in the mobile application or the backend. Every functional and non-functional requirement of Chapter 3 is traced to a test that validates it. The system is therefore not merely believed to work — it is demonstrated to work, repeatably and against the same public deployment that any user would reach.

Chapter 11 Deployment and Operations

A system that runs only on a developer's laptop is a demonstration, not a product. A central goal of this project was to deploy Keep It Green publicly so that anyone can try it. This chapter describes the deployment architecture, the hosting and database, the securing of traffic with HTTPS, the handling of configuration and secrets, the build and packaging of the components, the distribution of the mobile application, the concrete challenges encountered during deployment and how they were resolved, and the ongoing operation of the system.

11.1 Deployment Architecture

The backend, together with the web application and kiosk it serves, is deployed to a cloud host; user devices and the machine reach it over HTTPS; and the database sits on a private network reachable only from within the host. Figure 11.1 shows the topology. The whole user-facing system deploys as a single unit, because the web and kiosk interfaces are served by the backend itself.

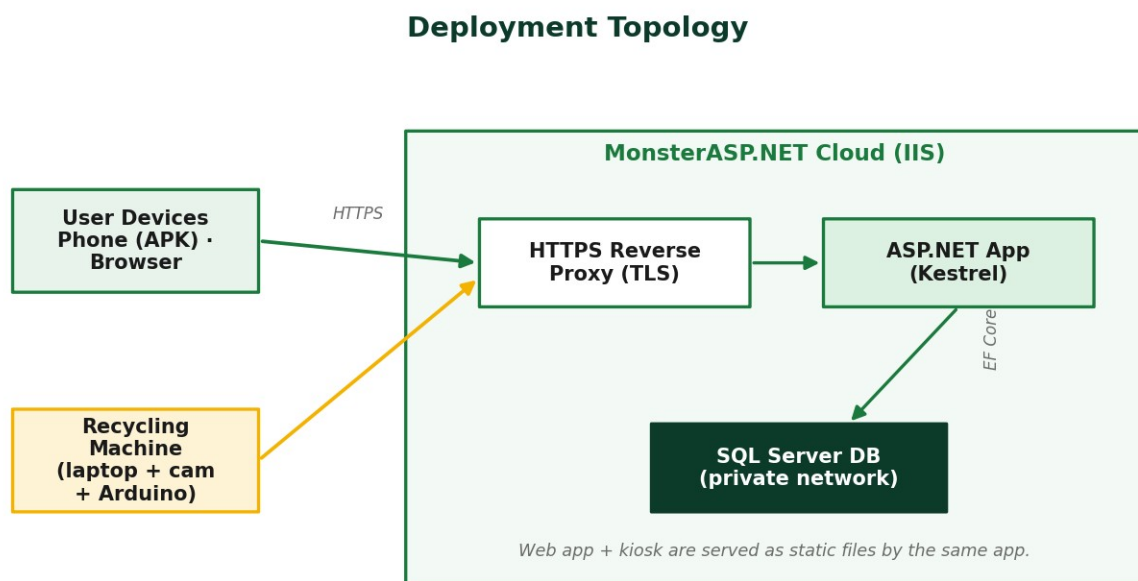


Figure 11.1: Deployment topology of the deployed system.

11.2 Hosting and Database

The system is hosted on a free ASP.NET hosting provider that runs the application under Internet Information Services and provides a free SQL Server database and automatic HTTPS. Choosing a free host was a deliberate demonstration that the system's running cost can be

negligible, which matters for the economic argument of Chapter 12. On first run, the application applies its database migrations automatically, creating the schema, and seeds the roles and the administrator account, so that no manual database setup is required. The database is reachable only from within the host's private network, which is why the automated tests are run against the public application rather than directly against the database.

11.3 Securing Traffic with HTTPS

Public exposure demands encrypted traffic. The host terminates HTTPS at a reverse proxy in front of the application, so the application is configured to trust the forwarded headers that the proxy adds, ensuring that it correctly recognizes requests as secure and constructs correct secure redirects rather than looping. With this in place, the site is served over HTTPS and the mobile application communicates with it securely. Resolving the initial ‘not secure’ warning was a matter of enabling this trust and the host's certificate, after which the padlock appears as expected.

11.4 Configuration and Secrets in Production

As described in Chapter 6, all secrets are supplied through configuration. In the deployed environment the connection string, the JSON Web Token signing key, the machine key, and the administrator credentials are set as environment configuration rather than being embedded in the build. Before the system was made public, all of these secrets were rotated to strong values, so that any value that may have appeared during development no longer grants access. This separation of secrets from code is what makes such rotation possible without recompiling or redeploying the application logic. Table 11.2 lists the configuration the deployed system reads from its environment.

Table 11.1: *Environment configuration of the deployed backend.*

Key	Purpose
ConnectionStrings:Default	SQL Server connection string
Jwt:Key / Issuer / Audience	Token signing and validation
Machine:ApiKey	Shared secret authenticating the machine
Seed:AdminEmail / AdminPassword	Seeded administrator credentials

11.5 Build, Packaging, and Containerization

The backend is published as a framework-dependent build and deployed to the host. A container image is also provided by way of a Dockerfile, so that the same backend can be run on any container platform if desired, which improves portability and reproducibility. The

mobile application is built as a release Android package. The web application requires no build step at all, being plain static files. This mix of packaging methods reflects the heterogeneity of the components while keeping each as simple as its technology allows.

11.6 Deployment Challenges and Resolutions

Deployment to a real host surfaced several concrete problems, each of which was diagnosed and resolved — an instructive part of the engineering, since these are exactly the issues a textbook omits. Table 11.1 records the most significant. The common thread is that a working local build is necessary but not sufficient; the environment differences of a real host — its packaging expectations, its startup behaviour, its file-locking, and its proxy — must each be accommodated.

Table 11.2: *Deployment challenges and their resolutions.*

Symptom	Root cause	Resolution
Upload archive would not extract	Archive written with backslash paths and satellite/design-time files	Repackage with forward-slash entries; strip culture and build-host folders
Application crashed on start (500.30)	Seeding used an unsupported password-reset path when the admin already existed	Rotate password via supported remove/add API, wrapped in a guard
Files locked during re-upload	The running application held its DLLs open	Use the host's app-offline procedure to stop, replace, then restart
Site reported 'not secure'	Application did not recognize the proxy's HTTPS	Trust forwarded headers; enable the host certificate

11.7 Operation and Maintenance

In operation the system requires little intervention: the backend runs continuously under the host, the database persists the data, and the automated migrations mean that a future schema change deploys without manual database work. The health of the deployment can be checked at any time through the live API documentation and by running the automated suites against the public endpoint, which together serve as a lightweight monitoring and regression mechanism. Routine maintenance amounts to redeploying updated builds through the host's procedure and rotating secrets periodically.

11.8 Rollback and Recovery

Because the backend deploys as a self-contained build, recovering from a bad release is straightforward: the previous build is redeployed through the same procedure, and because the database schema evolves only through additive migrations, a rollback of the application does not strand the data. The host's app-offline procedure provides a clean point at which to swap

builds without the file-locking problems encountered earlier. For a production deployment, scheduled database backups and a staging environment for pre-release validation would be added; at prototype scale, the ability to redeploy a known-good build within minutes is sufficient.

11.9 Cost of Operation

A deliberate outcome of the deployment choices is that the running cost of the system is essentially zero at prototype scale, which underpins the economic argument of Chapter 12. Table 11.3 summarizes the cost profile. The software stack is free and open-source; the hosting and database are on a free tier; and the only real expenditure is the inexpensive machine hardware, itself dominated by a computer that a production version could replace with a low-cost single-board device.

Table 11.3: *Cost profile of the deployed system (prototype).*

Item	Cost	Note
Software stack	Free	.NET, Flutter, YOLOv8, all open-source
Hosting + database	Free tier	Public HTTPS included
Machine hardware	Low (commodity)	Camera, Arduino, servos; laptop dominates
Rewards	Vendor-funded	Not a system cost; funded by partners

Chapter 12 Results and Discussion

This chapter steps back from the individual components to assess the system as a whole. It reviews the achievement of the objectives set in Chapter 1, summarizes the capabilities demonstrated, places the artificial-intelligence results in perspective, compares the system with the existing solutions surveyed in Chapter 2, considers its environmental and economic dimensions, and honestly states its limitations and the lessons learned.

12.1 Achievement of Objectives

Every objective defined in Chapter 1 was met. Table 12.1 maps each objective to its outcome. The headline results are that a working reverse-vending machine recognizes plastic and aluminium automatically; the detector reaches a mean Average Precision of 0.928, exceeding the 0.90 target, while running well within the one-second real-time budget; a secure, layered backend manages the full domain; mobile, web, and kiosk interfaces deliver the reward experience; a hardware sorter is integrated; and the entire system is deployed publicly and validated end-to-end with forty-eight passing checks.

Table 12.1: *Objectives and their outcomes.*

Objective	Outcome
Build an AI reverse-vending machine	Achieved — working prototype recognizes and sorts
Detector mAP@0.5 $\geq$ 0.90, real-time	Achieved — 0.928 mAP; ~87 ms CPU inference
Reject non-recyclables without an ‘other’ class	Achieved — confidence-threshold open-set rejection
Secure, layered backend for the full domain	Achieved — deployed .NET API, 48/48 tests pass
Safe multi-user session model	Achieved — ATM model, isolation tests pass
Mobile, web, and kiosk reward experience	Achieved — all three delivered and branded
Integrate a hardware sorter	Achieved — Arduino + servos via serial protocol
Deploy publicly and validate end-to-end	Achieved — live on the public internet over HTTPS

12.2 Capabilities Demonstrated

Beyond meeting the objectives individually, the project demonstrates that the parts cohere into a genuine system. A user really can approach the machine, pair their phone, insert a bottle and a can, watch twenty-five points appear live, finish the session, and redeem those points for a vendor code that is revealed only to them — and an administrator really can provision machines and rewards and read the resulting reports — all against a single deployed backend that anyone can reach. This working integration is the project's principal result, and it distinguishes the work from a collection of separately-impressive modules.

12.3 The AI Results in Perspective

The detector's performance deserves emphasis because it is what makes the whole concept viable without deposit legislation or barcodes. A mean Average Precision of 0.928 on unseen data, with negligible confusion between the two materials and with the smaller aluminium class actually outperforming plastic, shows that a carefully-engineered graduation-scale model can reach recognition quality sufficient for real use. The open-set rejection scheme, validated by the clean separation of the confidence distribution, is what turns that recognition into a machine that can be trusted not to reward the wrong thing. These results validate the central technical bet of the project.

12.4 Comparison with Existing Solutions

Against the alternatives surveyed in Chapter 2, Keep It Green occupies a distinctive and advantageous position, summarized again for outcomes in Table 12.2. It delivers the immediate reward that drives near-total return in deposit schemes, but without requiring the legislation those schemes depend on; it performs the physical verification and sorting that app-based schemes lack; and it does so at a small fraction of the cost of a commercial reverse-vending machine, on commodity hardware, independent of barcodes. Its trade-off — probabilistic visual recognition rather than deterministic barcode reading — is precisely the one the AI module was engineered to manage.

Table 12.2: *Keep It Green versus existing approaches (outcome view).*

Capability	Commercial RVM	Recycling app	Keep It Green
Immediate reward	Yes	Yes	Yes
Physical sorting	Yes	No	Yes
Works without deposit law	No	Partly	Yes
Works without barcodes	No	No	Yes
Low cost	No	Yes	Yes

12.5 Environmental Impact

The purpose of the system is environmental, and its potential impact scales with adoption. Every bottle and can returned through the machine is diverted from landfill or the environment into a recycling stream, and each recovered aluminium can in particular represents a large energy saving relative to primary production. Just as importantly, by attaching an immediate reward to the individual act of returning a container — the mechanism proven to lift return rates above ninety-five per cent in deposit-scheme countries — the system targets the behavioural bottleneck rather than merely providing another bin. At the scale of a university, a

campus, or a chain of venues, a network of such machines could measurably raise local recovery rates while generating data on recycling behaviour that is itself valuable.

12.6 Economic and Business Model

The system is designed to be economically self-sustaining. Its running cost is low by construction: it is built on free and open-source software, deployed on free hosting, and runs on inexpensive commodity hardware. Its rewards are funded not by a legislated deposit but by partner vendors, who gain footfall, brand exposure, and a demonstrable sustainability association in exchange for the promotional offers they provide. The user receives real value, the vendor receives marketing value, the host venue receives a sustainability amenity, and the environment receives recovered material — a model in which every party benefits. Scaling the vendor network and piloting machines at real venues are the natural next commercial steps, noted in Chapter 13.

12.7 Limitations

The project has honest limitations. Every training label is a real, human-drawn box, so localization is sound; the residual weakness is instead the plastic class, whose recall (0.76) trails aluminium's (0.97) because plastic bottles are far more visually varied. Collecting more plastic data, especially from the machine's own camera, is the clearest remedy. Rejection uses a single global confidence threshold rather than a per-class one, which is simple but not optimal. The physical machine is a supervised prototype rather than a hardened, weather-sealed, tamper-resistant unit, and it has no weight or fill sensing. And the model has not yet been trained on images from the machine's own camera, so a domain gap remains between the training data and the deployment view. None of these undermines the demonstrated system, but each defines a direction for improvement.

12.8 Lessons Learned

Several lessons emerged from the work. The first is that integration is where the real difficulty lies: each component was tractable alone, but making the machine, the backend, the phone, and the browser cooperate correctly — especially around the shared-machine session model — was the hardest and most valuable part, and building and testing end-to-end continuously was what made it manageable. The second is that data strategy dominates model architecture: the decisions to fuse four datasets, to reject out-of-class items by confidence, and to augment aggressively against imbalance mattered more to the final accuracy than any fine-tuning of the network. The third is that deploying to a real, constrained host teaches things a local build

never will, from packaging quirks to proxy behaviour. These lessons are themselves a result of the project.

12.9 SWOT Analysis

It is useful to summarize the system's position as a strengths, weaknesses, opportunities, and threats (SWOT) analysis, which draws together the discussion above into a single strategic view (Table 12.3).

Table 12.3: *SWOT analysis of Keep It Green.*

Strengths	Weaknesses
Works without deposit law or barcodes; low cost; complete and deployed; strong detection accuracy; instant phone reward.	Supervised prototype hardware; domain gap to the machine camera; single global confidence threshold; aluminium has least data.
Opportunities	Threats
National policy push toward 60% recycling; large young, mobile population; vendor-funded reward network; recycling-data analytics.	Fraud attempts on an unsupervised unit; hardware wear and vandalism; dependence on a partner-vendor ecosystem; network outages.

Chapter 13 Conclusion and Future Work

This final chapter summarizes what the project achieved, draws the overall conclusion, and sets out a concrete programme of future work.

13.1 Summary

Keep It Green set out to supply the reward layer that is missing from recycling in Egypt, using artificial intelligence in place of the deposit legislation and barcode infrastructure that formal schemes depend upon. It delivered a complete, integrated, and publicly deployed system: an AI reverse-vending machine that recognizes plastic bottles and aluminium cans with a mean Average Precision of 0.928 and sorts them physically; a secure, layered .NET backend that manages users, machines, sessions, points, and rewards; a Flutter mobile application, a JavaScript web application, and an on-machine kiosk that together deliver an ATM-style reward experience; and a low-cost Arduino-driven sorting mechanism. The system was validated end-to-end with forty-eight passing automated checks and deployed live over HTTPS for anyone to try.

13.2 Conclusion

The project demonstrates that the behavioural bottleneck in recycling — the absence of an immediate, convenient, tangible reward — can be addressed with accessible technology rather than legislation, and that commodity hardware and a carefully-engineered object detector are sufficient to build a reverse-vending machine that works without deposits or barcodes. In meeting all of its objectives and exceeding its accuracy target while running in real time and deployed on the public internet at negligible cost, Keep It Green shows that a graduation-scale team can produce not merely a proof of concept but a working, coherent, and useful system. The result is a foundation on which a real deployment could be built.

13.3 Future Work

A clear programme of work would take the system from a validated prototype toward a deployable product:

29. Close the domain gap by collecting several hundred labelled images from the machine's own camera and fine-tuning on them, which is expected to raise field accuracy and especially aluminium recall.
30. Broaden the materials to include glass and cartons, and adopt per-class confidence thresholds for more precise rejection.

31. Add a weight or fill sensor to deter fraud, reject empty or partial items, and signal when a bin is full.
32. Harden the enclosure for unsupervised, outdoor operation — weather-sealing, tamper resistance, and a low-cost single-board computer in place of the laptop.
33. Add offline queuing on the machine so that transactions are preserved and later reconciled through brief network interruptions.
34. Grow the partner-vendor network and the reward economy, and pilot several machines at universities, malls, and corporate campuses.
35. Provide anonymized recycling analytics to host venues and sustainability programmes, turning the network's data into a further source of value.

Pursuing this programme would transform Keep It Green from a working graduation project into a scalable contribution to the circular economy — which was the aspiration behind it from the beginning.

13.4 Closing Remarks

Keep It Green began as a simple observation — that people recycle when it is easy and rewarding, and do not when it is neither — and ended as a complete, deployed system that acts on that observation. Along the way the team learned that the hardest and most valuable engineering lies not in any single component but in making many different technologies cooperate correctly and securely, and that a careful data strategy and honest testing matter more than cleverness. The system that resulted is modest in scale but complete in scope, and it works: it recognizes, it sorts, it rewards, and it does so for anyone who cares to try it.

If the project encourages even a few more bottles and cans to be returned rather than discarded, and if it demonstrates to others that such a system can be built affordably and without waiting for legislation, then it will have served its purpose. The team is proud to submit it as the culmination of its undergraduate study.

References

- [1] The National, “One million bottles of water bought every minute around the world,” 2024.
- [2] EcoWatch, “The world buys a million plastic bottles per minute,” 2024.
- [3] United Nations Environment Programme, “Beat Plastic Pollution — facts and figures,” 2023.
- [4] OECD, “Global Plastics Outlook: only 9% of plastic waste is recycled,” 2022.
- [5] Eunomia, “Aluminium beverage can recycling rates,” 2023.
- [6] The Aluminum Association, “Recycling — the value and energy savings of aluminium,” 2024.
- [7] Egyptian Streets, “Egypt accelerates waste reforms, targets 60% recycling by 2027,” 2025.
- [8] Ministry of Environment (Egypt), “National Solid Waste Management Programme,” 2024.
- [9] TOMRA, “Deposit Return Schemes in Europe: Germany and Norway,” 2024.
- [10] Reloop Platform, “Global Deposit Book: return rates by country,” 2023.
- [11] Infinitum, “Norway's deposit return system — annual report,” 2023.
- [12] Bower, “How the Bower recycling reward app works,” 2024.
- [13] M. Yang and G. Thung, “Classification of Trash for Recyclability Status (TrashNet),” Stanford, 2016.
- [14] A. Kurmanov, “Drinking Waste Classification,” Kaggle, 2021.
- [15] Roboflow Universe, “Beverage Containers,” “YOLO Waste Detection,” and “Can, Bottle & Pack Detection” datasets, 2023–2024.
- [16] Roboflow Universe, “Bottle and can detection datasets,” 2023.
- [17] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” CVPR, 2016.
- [18] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023.
- [19] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” NeurIPS, 2015.
- [20] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” ECCV, 2014.
- [21] T.-Y. Lin et al., “Feature Pyramid Networks for Object Detection,” CVPR, 2017.
- [22] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization (AdamW),” ICLR, 2019.
- [23] Z. Zheng et al., “Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression,” AAAI, 2020.
- [24] Microsoft, “ASP.NET Core documentation,” 2024.
- [25] Microsoft, “Entity Framework Core documentation,” 2024.
- [26] Microsoft, “ASP.NET Core Identity and JWT bearer authentication,” 2024.
- [27] Google, “Flutter documentation,” 2024.
- [28] Google, “ML Kit barcode scanning,” 2024.
- [29] Arduino, “Servo library reference,” 2024.

[30] Internet Engineering Task Force, “RFC 7519: JSON Web Token (JWT),” 2015.

[31] ONNX, “Open Neural Network Exchange — format and runtime,” 2024.

Appendix A — API Endpoint Reference

This appendix lists the principal HTTP endpoints exposed by the backend. Authentication is shown as Public (no token), JWT-User, JWT-Admin, or Machine-Key. All request and response bodies are JSON.

Table A.1 — Backend API endpoints

Method	Path	Auth	Description
POST	/api/User/Register	Public	Create a user account
POST	/api/User/Login	Public	Authenticate; returns a JWT
GET	/api/User/Coins	JWT-User	Current point balance
POST	/api/Otp/Generate	JWT-User	Generate a pairing code
POST	/api/Otp/Pair	JWT-User	Pair the user to a code
POST	/api/Otp/EndSession	JWT-User	End the user's active session
POST	/api/Detection/session/start	Machine-Key	Start a machine session; returns code
GET	/api/Detection/status	Machine-Key	Whether a code is paired / active
POST	/api/Detection/submit	Machine-Key	Submit an accepted detection
GET	/api/Detection/total	Machine-Key	Points earned in a session
GET	/api/Transaction/User	JWT-User	The user's transaction history
GET	/api/Transaction/Admin/{machineId}	JWT-Admin	A machine's transaction report
GET	/api/Machine	JWT	List machines
POST	/api/Machine	JWT-Admin	Add a machine
PUT	/api/Machine/{id}	JWT-Admin	Update a machine (availability)
DELETE	/api/Machine/{id}	JWT-Admin	Delete a machine
GET	/api/Promo	JWT	List reward promotions
POST	/api/Promo	JWT-Admin	Create a promotion
POST	/api/Redemption/redeem	JWT-User	Redeem points for a promo code

Appendix B — Key Code Listings

B.1 Dataset descriptor (data.yaml)

The two-class dataset descriptor supplied to the training framework:

```
path: ./keep_it_green
train: images/train      # 10327 images
val:   images/val        # 978 images
test:  images/test       # 592 images
nc: 2
names:
  0: plastic_bottle
  1: aluminum_can
```

B.2 Detection submission (request and response)

A machine submits an accepted detection and receives the updated session total:

```
POST /api/Detection/submit
X-Machine-Key: <machine secret>
Content-Type: application/json

{ "code": "482913", "itemType": "plastic_bottle" }

HTTP/1.1 200 OK
{ "accepted": true, "points": 10, "sessionTotal": 10 }
```

B.3 Session-end detection on the machine (Python)

The machine polls the session status and returns to idle when the user finishes on their phone:

```
last_check = 0; ended = 0
while not finished:
    now = time.time()
    if now - last_check > 2.5:
        paired, _ = session_status(active_code)
        ended = ended + 1 if not paired else 0
        if ended >= 2:          # two consecutive 'not paired'
            finished = True    # user ended the session
        last_check = now
```

B.4 Service-worker cache versioning (JavaScript)

The web app's service worker is versioned so updates replace stale caches:

```
const CACHE = 'kig-v7';
self.addEventListener('activate', e => e.waitUntil(
  caches.keys().then(keys => Promise.all(
    keys.filter(k => k !== CACHE).map(k => caches.delete(k))))));
```

Appendix C — Database Schema

The principal tables of the database, expressed as simplified data-definition statements. Keys and foreign keys are indexed; timestamps are stored in UTC.

```
Users(Id PK, Email, PasswordHash, Points INT, Role)
Machines(Id PK, Name, Location, IsAvailable BIT)
Otps(Id PK, Code CHAR(6), UserId FK->Users NULL,
     MachineId FK->Machines, Status, ExpiresAt DATETIME)
Transactions(Id PK, UserId FK->Users, MachineId FK->Machines,
             ItemType, Points INT, CreatedAt DATETIME)
Vendors(Id PK, Name)
Promos(Id PK, VendorId FK->Vendors, Code, Cost INT)
Redemptions(Id PK, UserId FK->Users, PromoId FK->Promos,
             CreatedAt DATETIME)
```

The migrations that create these tables are applied automatically at first start, and the roles and default administrator are seeded at the same time, as described in Chapter 6.

Appendix D — Installation and User Manual

D.1 Running the Backend

The deployed backend is already live and requires no installation to use. To run it locally for development, restore and run the .NET project; the database schema and seed data are created automatically on first start:

```
dotnet restore
dotnet run --project Recycle
# Swagger UI served at /swagger
```

D.2 Installing the Mobile Application

Transfer the release Android package to an Android device and install it (allowing installation from the file), then open the app, register an account, and log in. The app targets the deployed backend by default; a different backend can be set in the in-app server settings.

```
flutter build apk --release
# output: build/app/outputs/flutter-apk/app-release.apk
```

D.3 Running the Machine

Connect the webcam and the Arduino, then start the machine program, which opens the full-screen kiosk, runs the detector on the camera, and drives the sorting servos:

```
python machine.py    # opens the kiosk; needs camera + Arduino
```

D.4 Using the System (End User)

36. At the machine, note the six-digit code (or the QR code) shown on the idle screen.
37. Open the app, tap Scan, and scan the QR code — or type the six-digit code.
38. Insert a plastic bottle or an aluminium can; watch the points appear on your phone.
39. Repeat for further items; then tap Finish to end the session.
40. Open Rewards, choose an affordable reward, and redeem it to reveal the code.

D.5 Administration

Sign in with an administrator account to reach the administrator interface, from which machines, vendors, and promotional codes are managed and per-machine reports are viewed. Administrative endpoints are refused to ordinary users.

Appendix E — Test-Case Catalogue

This appendix consolidates the automated end-to-end checks summarized in Chapter 10. Every listed case is automated and passes against the live deployed backend.

Table E.1 — Consolidated test cases

ID	Case	Expected	Result
C-01	Register new user	201 / account created	Pass
C-02	Login	200 / JWT returned	Pass
C-03	Initial balance	0 points	Pass
C-04	Start session	Unique code, unpaired	Pass
C-05	Pair user	Status paired, bound to user	Pass
C-06	Submit plastic	+10, total 10	Pass
C-07	Submit aluminium	+15, total 25	Pass
C-08	History	Both items listed	Pass
C-09	Redeem reward	Points deducted; code revealed	Pass
N-01	Invalid item type	Rejected	Pass
N-02	Submit without machine key	Rejected	Pass
N-03	Over-redeem	Rejected	Pass
A-01	Admin role in token	Role = Admin	Pass
A-02	Machine CRUD	Add/toggle/report/delete OK	Pass
A-03	User hits admin endpoint	403 Forbidden	Pass
I-01	Two sessions unique codes	Codes differ	Pass
I-02	Correct attribution	Each credited own items	Pass
I-03	No hijacking	Paired code not re-pairable	Pass

Appendix F — Hardware Bill of Materials

The prototype is built from inexpensive, commodity components. Costs are indicative and illustrate the low hardware cost of the concept; the computer dominates the cost and can be replaced by a low-cost single-board computer in production.

Table F.1 — Prototype bill of materials

Component	Qty	Indicative cost	Notes
Laptop / mini-PC	1	(existing)	Runs detection and kiosk
USB webcam	1	Low	Captures item images
Arduino board	1	Low	Drives the sorting servos
Servo motor	2	Very low	One gate per compartment
Enclosure + bins	1	Fabrication	3D-printed / CNC, brand-coloured
Cabling and mounts	—	Very low	USB, power, brackets

The actuation electronics — the microcontroller, servos, and cabling — total only a few dollars, which is central to the economic argument of Chapter 12.

Appendix G — Project Management

This appendix summarizes how the project was managed: the development phases, the tools used, the division of work, and the way risks and integration were handled. It complements the methodology of Chapter 1 and the team responsibilities of Table 1.2.

G.1 Development Phases

The work proceeded in incremental phases, each producing a demonstrable increment of the system. Table G.1 outlines the phases and their principal outputs. The ordering was chosen so that the backend and data model — on which everything else depends — were established first, the artificial-intelligence and client work proceeded in parallel against that stable core, and hardening and public deployment came last.

Table G.1 — Development phases

Phase	Focus	Principal output
1 · Foundations	Backend data model, auth, core flow	Running API with sessions and points
2 · Intelligence	Dataset fusion, model training, evaluation	YOLOv8 detector at 0.928 mAP
3 · Clients	Mobile, web, and kiosk interfaces	Reward experience on all surfaces
4 · Hardware	Arduino sorting, machine integration	Working reverse-vending prototype
5 · Hardening	Security, deployment, testing	Live public system, 48/48 tests

Project Timeline (indicative)

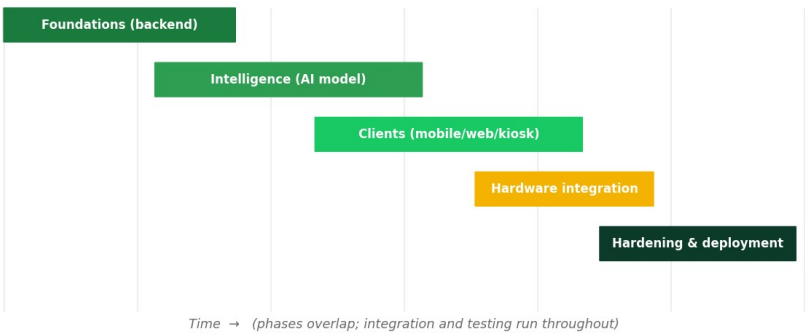


Figure G.1 — Indicative project timeline; phases overlap and integration runs throughout.

G.2 Tools and Environment

The team relied on a small, standard toolset, summarized in Table G.2. Version control coordinated parallel work; a shared, deployed backend meant every member developed against the same live system; and the automated test suites acted as a safety net that caught regressions across the many moving parts.

Table G.2 — Development tools

Area	Tools
Backend	.NET 10, Entity Framework Core, SQL Server, Swagger
AI	Python, Ultralytics YOLOv8, PyTorch, cloud GPUs (T4)
Mobile	Flutter, Dart, Android toolchain
Web / kiosk	JavaScript, HTML, CSS, pywebview
Hardware	Arduino IDE, Servo library
Collaboration	Version control, shared backend, automated tests

G.3 Division of Work and Integration

As set out in Chapter 1, responsibilities were divided by component — artificial intelligence and hardware, mobile, front-end, and back-end — while integration, testing, and deployment were shared. The interfaces between components were the API contract, which was agreed early and documented live, so that members could work independently against a stable contract and integrate continuously rather than in a single risky step at the end. This discipline is what allowed a six-person team to build and deploy a system spanning five very different technologies within the time available.

G.4 Risk and Change Management

Risks were tracked and mitigated as described in Chapter 3, and the iterative method meant that changes — a revised session model, a corrected camera configuration, a hardened deployment — were absorbed incrementally rather than destabilizing a monolithic plan. The most significant mid-project change, the diagnosis and correction of the camera defect described in Chapter 7, illustrates the value of the approach: because the system was continuously integrated and testable, the problem was isolated to a single configuration entry and fixed without disrupting the rest of the work.

Appendix H — Glossary

This glossary defines the principal technical terms used in this document, for readers who may be less familiar with them.

Table H.1 — Glossary of terms

Term	Definition
Object detection	Locating and classifying objects in an image, giving a box, a label, and a confidence for each.
Bounding box	The rectangle that marks where an object is in an image.
Confidence	A score, between 0 and 1, expressing how certain the model is about a detection.
Open-set rejection	Refusing inputs that do not belong to any trained class, here by requiring a minimum confidence.
Transfer learning	Adapting a model pre-trained on a large general dataset to a new, narrower task.
Data augmentation	Synthetically varying training images (flips, colour shifts, mosaics) to improve robustness.
mAP	Mean Average Precision — the standard headline accuracy metric for detectors.
IoU	Intersection over Union — the overlap between a predicted and a true box.
Reverse-vending machine	A machine that accepts a used container and rewards the depositor.
Deposit-return scheme	A system charging a refundable deposit on containers, reclaimed on return.
JWT	JSON Web Token — a signed token that authenticates a client on each request.
REST API	A web interface addressing resources by URL and using HTTP verbs and JSON.
Migration	A versioned, code-defined change to the database schema, applied automatically.
Progressive Web App	A web app installable to the home screen, with offline caching.
Serial protocol	A simple byte-level convention for the computer to command the microcontroller.
Stability buffer	Requiring a detection to persist for an interval before it is accepted, to avoid misfires.